

Machine Learning Project Report

UR3 Cobotops Dataset

조원 :

- 인공지능학과 202521328 김택명
- 인공지능학과 202521330 남궁도현
- 인공지능학과 202521338 유현성

분석 코드 및 개별 보고서 저장소:

GitHub - kzming2007/UR3_Cobot_ML

Contribute to kzming2007/UR3_Cobot_ML development by creating an account on GitHub.

https://github.com/kzming2007/UR3_Cobot_ML

kzming2007/
UR3_Cobot_ML



1 Contributor
0 Issues
0 Stars
0 Forks

목차

1. 프로젝트 개요
2. 탐색적 데이터 분석 (EDA)
3. Regression — “그리퍼 전류를 예측할 수 있는가?”
4. Classification — “로봇 고장을 예측할 수 있는가?”
5. Clustering — “로봇은 몇 가지 상태로 움직이는가?”
6. 종합 결론

1. 프로젝트 개요

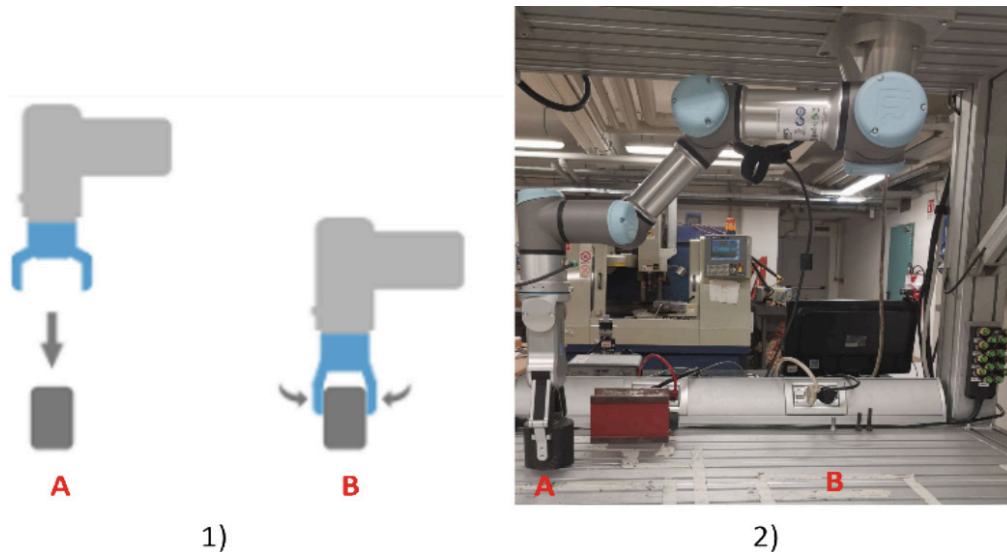
1.1. 데이터셋 출처 및 기본 구성

- 공식 출처: UCI Machine Learning Repository의 **UR3 CobotOps Dataset** (Cobot Operating Logs)
 - <https://archive.ics.uci.edu/dataset/963/ur3+cobotops>
- 관련 논문: M. Tyrovolas, C. Stylios, K. Aliev, and D. Antonelli, "Leveraging Information Flow-Based Fuzzy Cognitive Maps for Interpretable Fault Diagnosis in Industrial Robotics," in Technological Innovation for Human-Centric Systems, L. M. Camarinha-Matos and F. Ferrada, Eds. Cham, Switzerland: Springer, 2024, pp. 98–110.
 - https://link.springer.com/chapter/10.1007/978-3-031-63851-0_6
- 연구 배경: 스마트 팩토리 공정 내에서 UR3 협동로봇이 물건을 집어 이동시키는 픽애플레이스 공정을 실시간 기록한 물리 센서 데이터셋이다.
- 데이터 규모: 총 7,409개의 데이터로 구성되어 있으며, 로봇 관절 6개(J0~J5)마다 부착된 전류, 속도, 온도 센서 데이터 18개, 출력변수 2개, 로봇 진행 정도를 나타내는 변수 3개로 총 24개의 변수를 가진다.
 - 출력 변수:
 1. 그리퍼 구동 전류 (**Tool_current**): 그리퍼가 물건을 파지할 때 발생하는 전류 수치

2. 시스템 고장 여부 (System_Failure): 안전 보호 정지(Protective Stop) 또는 파지 상실(Grip Lost) 에러 발생 여부를 나타내는 이진 변수

1.2. 데이터 측정 방법

위의 이 데이터셋을 수집한 논문에 따르면 데이터 수집을 위해 OnRobot RG6 그리퍼를 장착한 6자유도 UR3 협동로봇(반복 정밀도 $\pm 0.1\text{mm}$, 작업 반경 500mm)을 사용하여 지점 A와 지점 B 사이의 픽애플레이스 공정을 수행하는 실험 환경을 구축했다.



OnRobot RG6 그리퍼를 탑재한 6자유도 UR3 협동로봇의 관절별(J0~J5) 구성 및 지점 A와 B 간의 픽애플레이스 작업 환경이다. J0은 천장에 가깝고 J5는 그리퍼에 가깝다.

• 실험 조건 및 하이퍼파라미터 변형:

실험 중 상태와 오작동 유발 요인을 다각도로 수집하기 위해 세 가지 프로세스 변수를 다르게 조합했다.

1. 하중(Workload): 1kg, 2kg, 3kg
2. 이동 속도(Movement Speed): 관절 최대 속도(210 rad/s)의 60%, 80%, 100%
3. 그리퍼 파지력(Gripping Force): 80N, 100N, 120N

위 조합에 따라 각 테스트 시나리오를 25회씩 반복 실행하여 '파지 상실(Grip Lost)' 및 '보호 정지(Protective Stop)' 고장 데이터를 측정했다.

• 측정 주파수 및 샘플링:

125 Hz의 고주파수로 데이터를 실시간 기록했다. 매 시간 걸음(Time-step)마다 관절 전류, 관절 속도, 관절 온도, 그리퍼 구동 전류, 그리고 고장 발생 플래그를 정합하여 데이터를 획득했다.

1.3. 주제 선정 이유

최근 산업 현장에서는 작업자와 동일한 공간에서 구동하는 로봇의 도입이 빠르게 늘고 있다. 작업자와 인접하여 작동하는 특성상, 로봇의 과부하나 기계적 결함을 실시간으로 감지하여 사고를 예방하는 기술이 매우 중요하다.

일반적으로 로봇 이상을 탐지하려면 고가의 카메라나 외부 토크 센서를 추가로 장착해야 한다. 본 프로젝트는 로봇이 구동하며 기본적으로 수집하는 내부 모터 센서 데이터(전류, 속도, 온도)만으로 로봇 상태를 진단하고 이상 동작을 예측할 수 있는지 확인하려는 현실적이고 경제적인 의문에서 출발했다.

UR3 Cobotops 데이터셋은 로봇이 천장에 매달려 중력을 지탱해야 하는 물리적 제약이 담겨 있고, 정상 가동 로그가 전체의 93%를 차지하여 데이터 불균형이 극심하며, 여러 관절의 힘이 비선형적으로 얽혀 있는 현장 데이터이다.

이처럼 다루기 까다로운 센서 로그 속에서 물리적 의미를 도출하고 패턴을 찾는 과정은 회귀, 분류, 군집화를 종합적으로 실험하고 각 알고리즘의 한계를 분석하기에 적합한 주제라고 생각했다.

1.4. 프로젝트 목표 및 주제- 로봇 센서 데이터로 무엇을 알 수 있는가?

본 프로젝트의 궁극적인 목표는 관절의 센서 데이터만으로 로봇 시스템의 전체 상태와 에러를 얼마나 정확히 설명하고 예측할 수 있는가를 확인하는 것이다.

선형, 비선형 알고리즘을 다각도로 적용하며 데이터 이면에 숨겨진 정보를 증명하는 과정에 집중했다.

주어진 세가지 머신러닝 Task별 주제는 다음과 같다.

● Regression (회귀) : “그리퍼 전류를 예측할 수 있는가?”

- 관절별 전류, 속도 데이터를 활용하여 로봇 말단 그리퍼의 구동 전류 예측을 수행
- 데이터의 비선형성과 분포 특성을 분석하여 연속형 예측의 한계 탐색
- 회귀에서 분류로 관점을 전환하여 재분석

● Classification (분류) : “로봇 고장을 예측할 수 있는가?”

- Protective Stop, Grip lost를 종합하여 시스템 에러 탐지 예측을 수행함
- 데이터 불균형을 해소하기 위해 SMOTE와 SMOTE에 언더샘플링을 진행 후 비교
- Confusion Matrix를 이용한 결과 비교

● Clustering (군집화) : “로봇은 몇 가지 상태로 움직이는가?”

- 정답 라벨 없이 센서 데이터만을 활용하여 로봇의 거시적인 물리적 운영 상태를 비지도 학습으로 군집화
- K-Means, DBSCAN, 계층적 군집화를 비교 실험, 단순 정량 지표에 매몰되지 않고 도메인 해석의 중요성을 파악
- 센서 데이터만으로 로봇의 유의미한 운영 상태 도출

flowchart TB

```
classDef task_node fill:#f3f4f6,stroke:#3b82f6,stroke-width:2px,color:#1f2937,font-weight:bold,rx:8,ry:8,word-break:keep-all;
```

```
classDef insight_node fill:#ffffeb,stroke:#f59e0b,stroke-width:2px,stroke-dasharray: 5 5,color:#92400e,font-weight:bold,rx:8,ry:8,word-break:keep-all;
```

```
classDef final_node fill:#ecfdf5,stroke:#10b981,stroke-width:3px,color:#065f46,font-weight:bold,rx:8,ry:8,word-break:keep-all;
```

```
subgraph SG1 ["STAGE 1. 회귀 분석 (Regression)"]
    direction LR
    A["Task 1. Regression<br/>그리퍼 전류 예측이<br/>가능한가?"]:::task_node
    B["Insight 1. 이산적 상태 규명<br/>(3가지 층으로 분포)<br/>→ 분류 문제로 패러다임 전환"]:::insight_node
    A -- "R2 0.58 한계 분석" --> B
end
```

```
subgraph SG2 ["STAGE 2. 분류 분석 (Classification)"]
    direction LR
    C["Task 2. Classification<br/>로봇 고장 예측이<br/>가능한가?"]:::task_node
    D["Insight 2. 핵심 피처 발굴<br/>(전류가 고장 예측의 핵심)<br/>→ 주요 전류 변수 추출"]:::insight_node
    C -- "RF 피처 중요도 분석" --> D
end
```

```
subgraph SG3 ["STAGE 3. 군집 분석 및 결론"]
    direction LR
    E["Task 3. Clustering<br/>로봇은 몇 가지 상태로<br/>움직이는가?"]:::task_node
    F["최종 결론 및 시너지<br/>'과부하 군집' 내<br/>실제 고장(PS)의<br/>14.34% 발생 증명"]:::final_node
    E -- "에러 라벨 교차 검증" --> F
end
```

SG1 -->|이진 분류 적용| SG2
SG2 -->|전류 변수 기반 군집화| SG3

2. 탐색적 데이터 분석 (EDA)

머신러닝 모델링에 앞서 데이터셋의 변수 구성, 결측치 현황, 물리적 신호 간의 상관관계 및 분포적 특성을 확인하고자 탐색적 데이터 분석을 수행했다. PPT 발표 자료에 수록된 4가지 핵심 시각자료를 바탕으로 분석한 결과는 다음과 같다.

2.1 데이터 결측치 및 변수 구조 분석 (자료 1, 2)

<class 'pandas.core.frame.DataFrame'> RangeIndex: 7409 entries, 0 to 7408 Data columns (total 24 columns): # Column Non-Null Count Dtype --- 0 Num 7409 non-null int64 1 Timestamp 7409 non-null object 2 Current_J0 7363 non-null float64 3 Temperature_T0 7355 non-null float64 4 Current_J1 7355 non-null float64 5 Temperature_J1 7355 non-null float64 6 Current_J2 7355 non-null float64 7 Temperature_J2 7355 non-null float64 8 Current_J3 7355 non-null float64 9 Temperature_J3 7355 non-null float64 10 Current_J4 7355 non-null float64 11 Temperature_J4 7355 non-null float64 12 Current_J5 7355 non-null float64 13 Temperature_J5 7355 non-null float64 14 Speed_J0 7355 non-null float64 15 Speed_J1 7355 non-null float64 16 Speed_J2 7355 non-null float64 17 Speed_J3 7355 non-null float64 18 Speed_J4 7355 non-null float64 19 Speed_J5 7355 non-null float64 20 Tool_current 7355 non-null float64 21 cycle 7409 non-null int64 22 Robot_ProtectiveStop 7355 non-null object 23 grip_lost 7409 non-null bool dtypes: bool(1), float64(19), int64(2), object(2) memory usage: 1.3+ MB					Num 0
					Timestamp 0
					Current_J0 46
					Temperature_T0 54
					Current_J1 54
					Temperature_J1 54
					Current_J2 54
					Temperature_J2 54
					Current_J3 54
					Temperature_J3 54
					Current_J4 54
					Temperature_J4 54
					Current_J5 54
					Temperature_J5 54
					Speed_J0 54
					Speed_J1 54
					Speed_J2 54
					Speed_J3 54
					Speed_J4 54
					Speed_J5 54
					Tool_current 54
					cycle 0
					Robot_ProtectiveStop 54
					grip_lost 0

데이터프레임의 요약 정보(`df.info()`)와 변수별 결측치 현황을 분석하여 데이터셋의 무결성을 점검했다.

- 변수 및 데이터 타입 구조 (`df.info()`):

전체 데이터셋은 7,409개의 행과 24개의 열로 구성되어 있다. 인덱스 및 시간 정보를 제외하면 대부분 관절 전류, 온도, 속도 신호를 나타내는 실수형(`float64`) 변수이다.

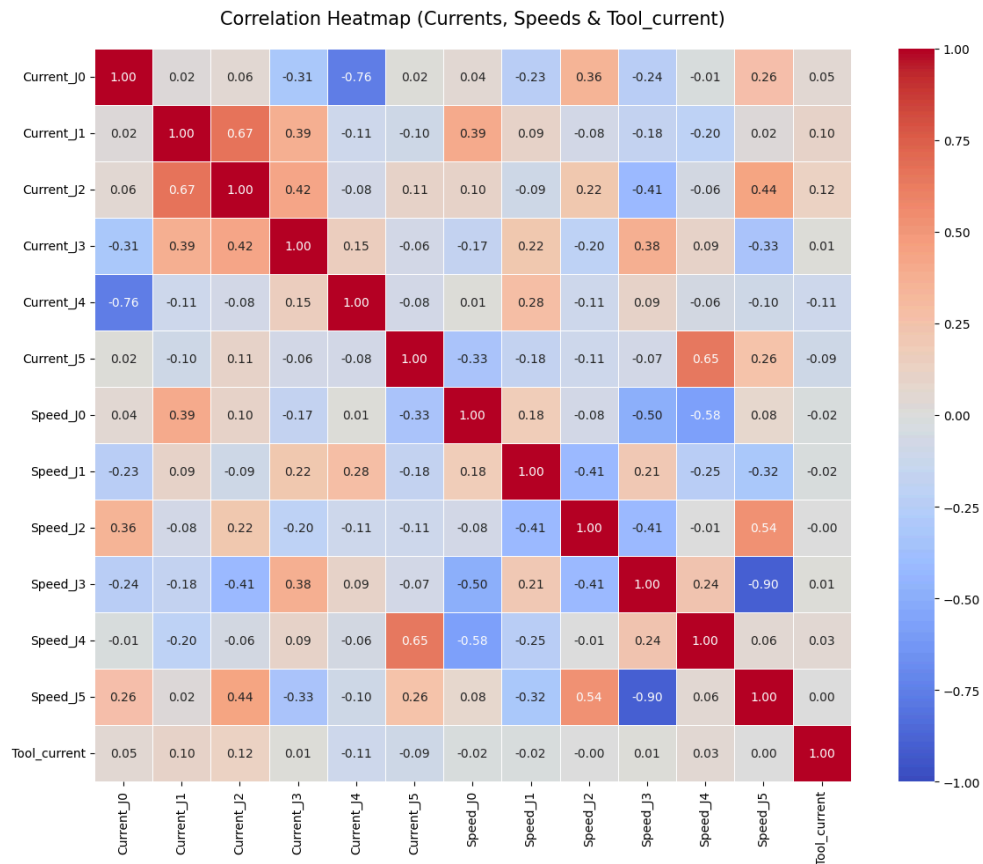
- 결측치(Missing Value) 분포:

- 분석 결과, 센서 데이터 수집 중 일부 신호의 통신 누락으로 인해 대부분의 수치형 변수(관절 전류, 온도, 속도, 그리퍼 전류 등)에서 **54개**의 결측치가 공통으로 관찰되었다.
- 예외적으로 0번 관절 전류(`Current_J0`)는 **46개**의 결측치를 가진다.

- 코드 실행 흐름에 따른 데이터 처리 순서:

1. 7,409행 24열의 CSV 원본 데이터를 데이터프레임으로 로드하고 데이터 타입을 탐색한다.
2. 데이터프레임 내 결측치가 발생한 행을 일괄 제거(`df.dropna()`)하여 머신러닝 알고리즘의 안정적 연산을 확보한다. 이 과정에서 54개 행이 제거되어 최종 **7,355행**으로 데이터가 정제된다.
3. 데이터 타입 변환 및 불리언 인코딩:
 - `object` 타입으로 로드된 `Robot_ProtectiveStop` 변수의 문자열 값(`'True'`, `'False'`)을 머신러닝 모델이 연산할 수 있도록 이진값(0과 1 또는 True/False)으로 타입 변환 및 매핑을 수행한다.
 - `grip_lost` 역시 통일된 이진 논리값(Boolean) 형태로 매핑한다.
4. 고장 상태를 정의하기 위해 두 에러 변수를 결합하여, `Robot_ProtectiveStop` 과 `grip_lost` 중 하나라도 발생하면 고장(`1`), 둘 다 정상이면 정상(`0`)을 부여하는 이진 분류 타겟인 `System_Failure` 변수를 생성한다.
5. 모델의 과적합 방지와 연산 속도 향상을 위해 기계적 물리량과 관련이 없는 단순 시퀀스 번호(`Num`), 시간 정보(`Timestamp`), 사이클 횟수(`cycle`) 변수를 분석 대상 피처에서 제외한다.

이렇게 정제된 데이터는 전류, 속도, 온도 등 변수마다 물리적 단위와 수치 범위가 상이하므로, 실제 모델 학습 직전에 `StandardScaler` 를 적용하여 평균 0, 분산 1의 표준화 데이터로 변환하여 사용한다.



2.2 관절 전류 및 속도 변수 간의 상관관계 히트맵 분석 (자료 3)

주요 입력 변수(Current, Speed)와 타겟 변수(`Tool_current`) 간의 피어슨 상관계수(를 히트맵으로 시각화하여 관절 간 동역학적 연관성을 분석했다.

- 주요 관절 전류 간의 상관관계:

- 어깨 관절(`Current_J1`)과 팔꿈치 관절(`Current_J2`) 간의 상관계수는 **0.67**로 강한 양의 상관관계를 나타낸다. 이는 로봇의 링크 자중과 물건 이송 시 발생하는 모멘트 하중을 두 대형 관절이 물리적으로 분담하여 지탱하고 있음을 나타내는 동역학적 증거이다.

- 베이스 관절(`Current_J0`)과 손목 관절(`Current_J4`) 간의 상관계수는 **-0.76**으로 뚜렷한 음의 상관관계를 보인다. 이는 로봇 팔이 회전 구동할 때 동역학적 균형을 유지하거나 케이블 하중을 상쇄하기 위해 두 관절의 모터 전류가 반대 방향으로 밀접하게 연동되고 있음을 뜻한다.
- **타겟 변수와 입력 변수의 무상관성:**
 - 예측 대상인 그리퍼 전류(`Tool_current`)는 다른 모든 관절 전류 및 속도 변수들과의 상관계수가 **최대 0.12**에 그칠 정도로 선형 상관관계가 매우 낮다.
 - 이처럼 독립변수와 종속변수 간의 선형 관계가 거의 존재하지 않기 때문에, 단순 다중 선형 회귀(OLS) 모델로는 그리퍼 전류를 전혀 예측할 수 없음을 알 수 있다. 따라서 비선형 관계를 학습할 수 있는 머신러닝 알고리즘 적용이 필수적이다.

2.3 그리퍼 전류의 시계열 산점도 및 삼봉(Tri-modal) 분포 (자료 4)

타겟 변수인 그리퍼 전류(`Tool_current`)의 전체 관측값을 시간 흐름(Data Index)에 따라 산점도로 매핑하여 분포적 특성을 확인했다.

• 이산적인 세 가지 전류 밴드 형성:

산점도 분석 결과, 그리퍼 전류는 연속적인 분포를 보이지 않고 가로 방향으로 뚜렷하게 구분되는 세 개의 기저 전류 대역을 형성하고 있다. 이는 로봇 그리퍼의 동작 모드가 물리적으로 명확히 구분되어 작동함을 나타낸다.

1. 제1대역 (0.08A ~ 0.09A 대역):

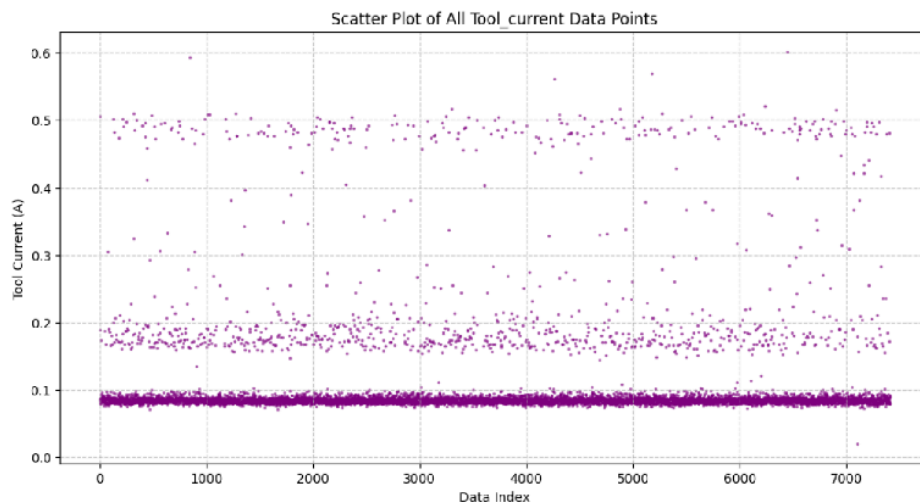
그리퍼가 물건을 잡지 않고 열린 채 대기하는 정적 상태 (Standby)

2. 제2대역 (0.17A ~ 0.20A 대역):

그리퍼가 닫히거나 열리기 위해 모터가 회전하며 가속하는 전이 상태 (Transition)

3. 제3대역 (0.49A ~ 0.50A 대역):

그리퍼가 목적 물체를 완전히 파지하여 일정한 압축 토크를 가하고 있는 상태 (Gripping)



• 머신러닝 모델링에 주는 시사점:

이러한 삼봉 분포 특성은 그리퍼 전류가 단순한 연속형 수치라기보다는 물리적 상태의 천이에 의존하는 불연속적 성격을 가졌음을 증명한다. 이로 인해 수치 자체를 예측하는 회귀 모델은 결정계수 R^2 한계에 봉착할 가능성이 높으며, 상태 분류나 군집화를 병행하는 분석이 동역학적으로 타당함을 시사한다.

더 자세한 EDA 과정은 아래의 보고서에 담겨있다.

https://github.com/kzming2007/UR3_Cobot_ML/blob/main/reports/EDA_report.pdf

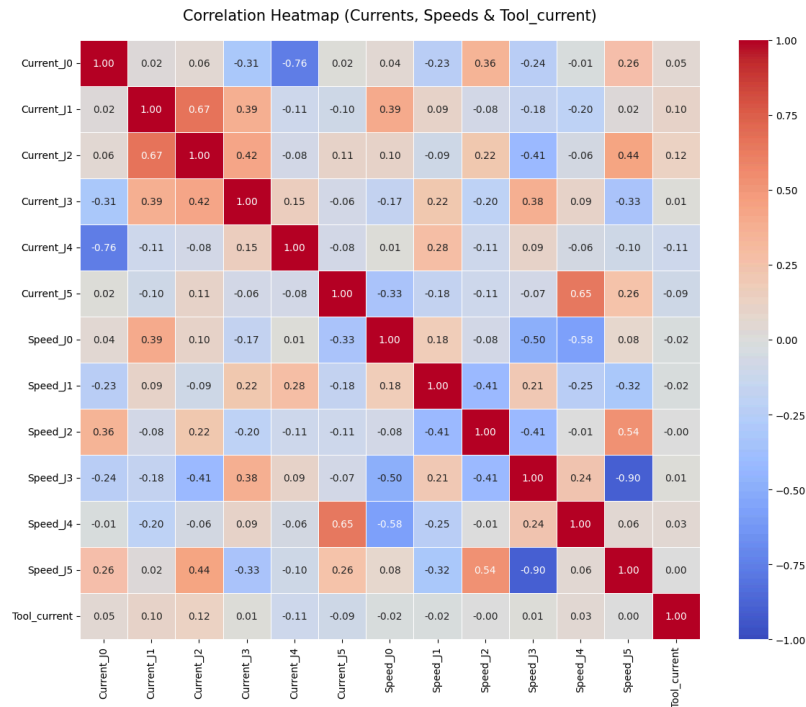
3. Regression — “그리퍼 전류를 예측할 수 있는가?”

3.1 태스크 정의 및 타당성

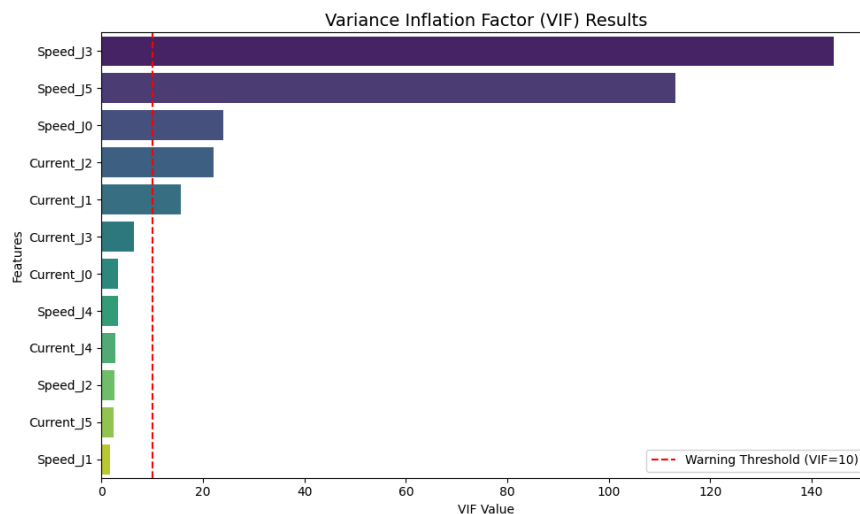
본 회귀 분석의 목표는 6개 관절의 전류와 속도 신호인 총 12개의 센서 데이터를 독립변수로 사용하여 그리퍼의 구동 전류인 Tool Current를 예측하는 것이다.

관절 센서만으로 그리퍼 전류를 정확히 예측할 수 있다면, 고가의 말단 센서 없이도 로봇 팔의 내부 데이터만으로 작업 상태를 포착할 수 있다고 할 수 있다.

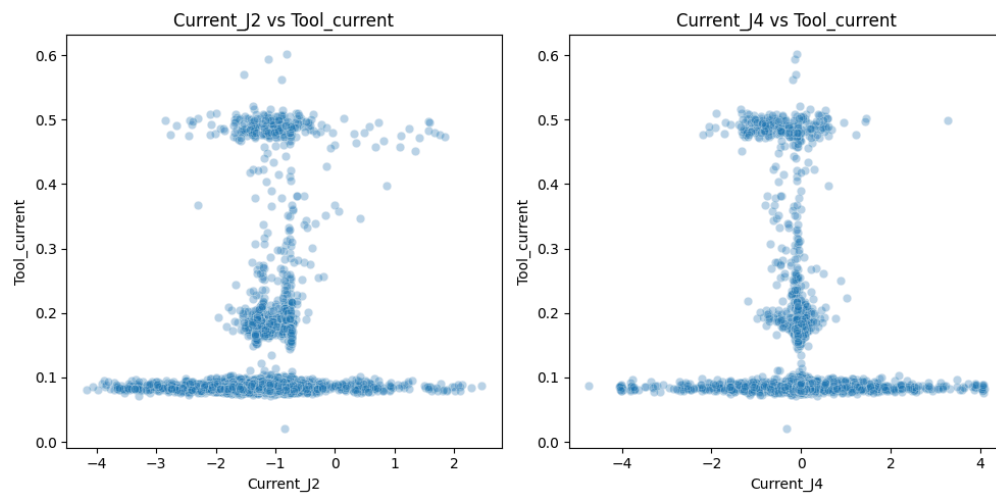
앞선 EDA 단계에서 이미 회귀 모델의 설계 도전 과제가 드러난다.



- 타겟 변수인 Tool Current와 모든 관절 변수 사이의 상관계수가 최대 0.12에 불과하여, 단순 선형 회귀의 예측력에 뚜렷한 한계가 예견된다.



- 독립변수들 사이에서 Speed J3과 J5의 다중공선성 지표인 VIF 수치가 110을 초과하고, Current J1과 J2의 VIF 역시 15를 초과하는 등 극심한 다중공선성이 관찰된다. 가중치 규제가 없는 선형 회귀를 적용하면 회귀 계수의 분산이 비정상적으로 팽창하여 모델의 신뢰도가 무너질 위험이 있다.



- 관절 전류(Current J2) 대비 Tool Current 산점도에서, 관절 전류가 넓은 범위로 변해도 그리퍼 전류는 0.08A 부근에 고정되어 있다가(수평 구간), 특정 관절 자세에서만 그리퍼 전류가 수직으로 치솟는(수직 기둥) 역 T자형 패턴이 관찰된다. 이는 선형 모델이 학습하기 어려운 구조이다.

3.2 Linear Regression

기본 코드 설정


```

# 1. 결측치 없는 클린 데이터 로드 및 분리
df_clean = df.dropna().copy()

# 입력 피쳐(X): 12개 조인트 피쳐
input_cols = [
    'Current_J0', 'Current_J1', 'Current_J2', 'Current_J3', 'Current_J4', 'Current_J5',
    'Speed_J0', 'Speed_J1', 'Speed_J2', 'Speed_J3', 'Speed_J4', 'Speed_J5'
]
X = df_clean[input_cols]

# 타겟(y): 그리퍼 전류
y = df_clean['Tool_current']

# 2. Train / Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. 비교할 스케일러 정의
scalers = {
    "StandardScaler": StandardScaler(),
    "MinMaxScaler": MinMaxScaler()
}

print("--- [최적 피벗] 그리퍼 전류(Tool_current) 스케일러별 회귀 결과 ---")

# 4. 루프를 돌며 스케일러별 학습 및 평가
for name, scaler in scalers.items():
    # 데이터 스케일링
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # 모델 학습 및 예측
    model = LinearRegression()
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    # 결과 출력
    print(f"\n[{name} 결과]")
    print(f"R² (결정계수): {r2_score(y_test, y_pred):.4f}")
    print(f"MSE (평균제곱오차): {mean_squared_error(y_test, y_pred):.6f}")

```

12개 관절 센서를 X로, 예측한 Tool Current를 Y로 넣고 Train과 Test셋을 8:2로 분할한다.

X_train에만 `scaler.fit_transform(X_train)` 을 사용하여 학습 데이터의 평균과 표준편차를 학습한 뒤, 그 기준에 맞추어 데이터를 변환한다.

X_test에 `scaler.transform(X_test)` 을 사용하여 새로운 평균과 표준편차를 구하지 않고, 앞서 X_train에서 학습한 기준을 그대로 가져와서 변환만 수행한다. 이는 테스트 데이터의 통계 정보가 학습 모델에 유출되는 데이터 누수 현상을 방지하여 일반화 성능의 신뢰성을 보장하기 위함이다.

순수 선형 회귀 및 스케일러 비교

아무런 변환 없이 12개 피쳐를 그대로 사용한 Linear Regression의 분석 결과는 $R^2 = 0.1047$, $MSE = 0.004917$ 이다.

```

--- [최적 피벗] 그리퍼 전류(Tool_current) 스케일러별 회귀 결과 ---

[StandardScaler 결과]
R² (결정계수): 0.1047
MSE (평균제곱오차): 0.004917

[MinMaxScaler 결과]
R² (결정계수): 0.1047
MSE (평균제곱오차): 0.004917

```

StandardScaler 와 MinMaxScaler 를 적용한 결과는 동일하다. 규제가 없는 선형 회귀의 수학적 특성상 입력 데이터 스케일링이 최종 성능에 영향을 미치지 않기 때문이다.

다항 회귀 피쳐 및 Ridge, Lasso, ElasticNet 규제

--- [최적화 결과] 선형 규제 모델 상위 10개 성능 ---

	Scaler	Model	Alpha	L1_Ratio	R2	MSE
Rank						
1	Standard	Ridge	0.0100	-	0.2340	0.004207
2	Standard	Ridge	0.1000	-	0.2340	0.004207
3	Standard	Ridge	1.0000	-	0.2340	0.004207
4	Standard	Ridge	10.0000	-	0.2340	0.004207
5	Standard	ElasticNet	0.0001	0.500000	0.2334	0.004210
6	Standard	ElasticNet	0.0001	0.200000	0.2334	0.004210
7	Standard	ElasticNet	0.0001	0.800000	0.2325	0.004215
8	Standard	Lasso	0.0001	-	0.2322	0.004217
9	Standard	Ridge	100.0000	-	0.2309	0.004224
10	Standard	ElasticNet	0.0010	0.200000	0.2299	0.004229

12개의 입력 피쳐를 2차 다항식으로 변환하여 90개의 교차 피쳐를 생성한 뒤, 규제 회귀 모델을 적용한다.

모델	R ²	MSE	핵심 해석
Poly + Ridge ($\alpha=0.01\sim10.0$)	0.234	0.0042	상위권 독점
Poly + Lasso	하위권	—	변수를 완전 제거하는 특성이 불리
Poly + ElasticNet	중위권	—	L1/L2 규제 혼합

Ridge 모델이 우수한 성능을 나타내는 것은, 90개로 늘어난 다항 교차 변수들이 특정 소수 변수에만 의존하지 않고 모든 변수가 미세하게나마 Tool Current 예측에 기여하고 있음을 뜻한다. 따라서 변수 가중치를 완전히 0으로 만드는 Lasso 모델은 본 데이터셋 구조에 불리하게 작용한다.

5-Fold 교차 검증

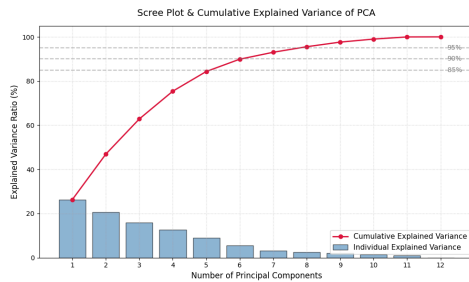
--- [최적화 완료] 교차 검증 기반 상위 10개 모델의 최종 Test 세트 성능 ---

	Scaler	Model	Alpha	L1_Ratio	CV Train R ²	Test R ²	Test MSE
Rank							
1	MinMax	Ridge	0.0100	-	0.2499	0.2298	0.004230
2	Standard	ElasticNet	0.0001	0.500000	0.2495	0.2334	0.004210
3	Standard	Ridge	10.0000	-	0.2495	0.2340	0.004207
4	Standard	ElasticNet	0.0001	0.800000	0.2495	0.2325	0.004215
5	Standard	ElasticNet	0.0001	0.200000	0.2494	0.2334	0.004210
6	Standard	Lasso	0.0001	-	0.2492	0.2322	0.004217
7	Standard	Ridge	1.0000	-	0.2487	0.2340	0.004207
8	Standard	Ridge	0.1000	-	0.2483	0.2340	0.004207
9	Standard	Ridge	0.0100	-	0.2482	0.2340	0.004207
10	Standard	ElasticNet	0.0010	0.200000	0.2479	0.2299	0.004229

5-Fold 교차 검증 적용 결과, MinMaxScaler 모델이 학습 데이터 교차 검증 R² 기준 1위를 기록했으나 학습 데이터와 평가 데이터 간의 오차가 0.0201로 다른 모델(0.014~0.015)에 비해 커 과적합이 의심된다.

따라서 실전에 투입할 최적 모델은 단순 순위가 아니라, 평가 데이터 성능이 우수하고 오차가 낮은 StandardScaler와 다항 피쳐 및 Ridge 규제를 조합한 모델로 판단된다.

주성분 분석(PCA) 통합 및 분리 대조 실험



[각 주성분(PC)의 분산 설명력 및 누적 설명력]			
Principal Component	Explained Variance (%)	Cumulative Variance (%)	
PC1	26.35	26.35	
PC2	20.57	46.92	
PC3	15.92	62.84	
PC4	12.57	75.41	
PC5	8.92	84.33	
PC6	5.56	89.89	
PC7	3.13	93.03	
PC8	2.51	95.54	
PC9	2.08	97.63	
PC10	1.38	99.00	
PC11	0.97	99.97	
PC12	0.03	100.00	

--- [PC 7 차원 축소 버전] Grid Search를 진행 중입니다 ---

--- [PC 8 차원 축소 버전] Grid Search를 진행 중입니다 ---

```

=====
=== [5단계] PCA (PC 7 vs PC 8) 하이퍼파라미터 최적화 모델 최종 비교 ===
Dimension      Model      Best Hyperparameters      R2 (Test)      MSE      RMSE
PCA (PC 7)     Ridge      {'model__alpha': 100.0}    0.022211      0.005370      0.073279
PCA (PC 7)     Lasso      {'model__alpha': 0.0001}    0.022120      0.005370      0.073283
PCA (PC 7)     ElasticNet {'model__alpha': 0.001, 'model__l1_ratio': 0.5} 0.021197      0.005375      0.073317
PCA (PC 8)     Ridge      {'model__alpha': 100.0}    0.029237      0.005331      0.073016
PCA (PC 8)     Lasso      {'model__alpha': 0.0001}    0.029115      0.005332      0.073020
PCA (PC 8)     ElasticNet {'model__alpha': 0.001, 'model__l1_ratio': 0.2} 0.028888      0.005333      0.073029
  
```

12개 피쳐 전체에 대해 주성분 분석을 수행한 결과, 7개의 주성분으로 데이터 분산의 93%를 설명할 수 있었으나 회귀 예측 성능은 오히려 하락하였다.

이는 주성분 분석이 타겟 변수와의 상관성을 배제하고 독립변수 자체의 분산만을 보존하기 때문이다. 특히 전류와 속도는 물리적 단위와 의미가 상이한 도메인 변수이므로, 이를 무분별하게 병합하면 변수 고유의 물리적 상호작용 정보가 유실된다.

--- [6단계] Current / Speed 독립 분할 PCA 기반 K-Fold 최적화 시작 ---

```

[최적 하이퍼파라미터]
{'model__alpha': 0.0001, 'model__l1_ratio': 0.2, 'preprocessor__curr_pca__pca__n_components': 5, 'preprocessor__spd_pca__pca__n_components': 5}

R2 (Test) : 0.083068
MSE       : 0.005036
RMSE      : 0.070962
  
```

이를 해결하기 위해 물리적 성향이 다른 Speed와 Current를 분리하여 각각 주성분 분석을 적용한 결과, 통합 방식 대비 약 4배 향상된 회귀 성능을 얻는다. 이는 데이터의 도메인 특성을 반영하여 차원 축소를 분리 수행하는 전처리가 통계적 한계를 효과적으로 보완할 수 있음을 시사한다.

3.3 Decision Tree & Random Forest

의사 결정 나무

다항식 변환과 규제를 적용해도 선형 회귀 R^2 가 0.23 수준에 머무름에 따라, 비선형 경계를 분할 학습할 수 있는 의사 결정 나무 모델을 도입한다.

```
--- [최종 결과] Decision Tree Regressor ---

[Train 데이터 평가]
R² (결정계수) : 0.6773
MSE : 0.002062
RMSE : 0.045414
MAE : 0.017672

[Test 데이터 평가]
R² (결정계수) : 0.3211
MSE : 0.003728
RMSE : 0.061061
MAE : 0.023271
```

GridSearchCV를 통한 최적 파라미터(max_depth=10, min_samples_split=5, min_samples_leaf=20) 학습 결과, $R^2 = 0.3882$ 를 기록하며 선형 모델 대비 성능이 개선된다. 그러나 학습과 테스트 성능 간 오차가 0.35에 달해 심각한 과적합 현상이 발생한다.

과적합의 원인은 데이터의 물리적 특성에서 기인한다. 입력 변수 중 상관성이 가장 높은 Current J2를 보면, 완전히 동일한 로봇 자세 값에서도 그리퍼 전류인 Tool Current가 0.08A(대기)와 0.49A(파지)로 상이하게 나타난다. 이처럼 입력값은 동일하나 출력 정답이 다중으로 존재하는 상태에서 의사결정나무가 무리하게 노드를 분기하면서 센서 노이즈까지 단순 암기하여 적합한 결과이다.

```
--- [모델 비교] 상위 5개 Decision Tree 성능 종합 ---
```

	순위	CV R²	Train R²	Test R²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.3987	0.5541	0.3665	0.1876	0.002850	0.003479	D=7, L=7, S=10
1	2	0.3985	0.5521	0.3669	0.1853	0.002863	0.003477	D=7, L=7, S=20
2	3	0.3953	0.6004	0.3430	0.2574	0.002554	0.003608	D=8, L=7, S=20
3	4	0.3931	0.6048	0.3493	0.2555	0.002526	0.003574	D=8, L=7, S=10
4	5	0.3855	0.5525	0.3292	0.2233	0.002860	0.003684	D=8, L=15, S=10

```
=====
--- [상세 분석] 1위 최적 모델 최종 평가지표 ---
=====

[Train 데이터 평가]
R² (결정계수) : 0.5541
MSE : 0.002850
RMSE : 0.053385
MAE : 0.023017

[Test 데이터 평가]
R² (결정계수) : 0.3665
MSE : 0.003479
RMSE : 0.058985
MAE : 0.025088
```

이에 따라 하이퍼파라미터를 재조정(max_depth=7, min_samples_split=20, min_samples_leaf=10)하여 확인해본 결과, Test R^2 0.3669, 성능 오차 0.1853으로 과적합 수준을 기존 대비 절반으로 완화할 수 있었다.

도메인 기반 파생 피쳐 개발: `is_stationary`

EDA 단계에서 포착한 속도 0.0 부근에서의 전류 수직 분포 특성을 모델에 학습시키기 위해, 6개 관절 속도의 절대값 합이 0에 수렴하면 True(1), 그렇지 않으면 False(0)를 반환하는 정지 여부 파생 변수 `is_stationary` 를 생성한다.

이 파생 피쳐는 나무 모델이 여러 하위 노드를 거쳐 정지 상태를 유추해야 하는 연산 비용을 절감하여, 최상단 루트 노드에서 한 번에 정지 및 구동 모드를 분기할 수 있도록 돕는다.

--- [모델 비교] 상위 10개 Decision Tree 성능 종합 ---

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.4256	0.6253	0.4442	0.1810	0.002395	0.003052	D=8, L=7, S=20
1	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=25
2	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=20
3	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=10
4	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=30
5	6	0.4169	0.6397	0.4697	0.1700	0.002303	0.002912	D=8, L=7, S=10
6	7	0.4168	0.5990	0.4201	0.1789	0.002563	0.003185	D=7, L=7, S=20
7	8	0.4164	0.5567	0.4273	0.1294	0.002833	0.003145	D=6, L=7, S=20
8	9	0.4150	0.5586	0.4342	0.1244	0.002821	0.003108	D=6, L=7, S=10
9	10	0.4149	0.6089	0.4391	0.1698	0.002500	0.003081	D=7, L=7, S=10

조건	Test R ²	성능 오차 (Gap)	Test MSE
파생 변수 도입 전 (의사결정나무 최고)	0.3669	0.1853	0.003477
파생 변수 도입 후 (의사결정나무 최고)	0.4697	0.1700	0.002912

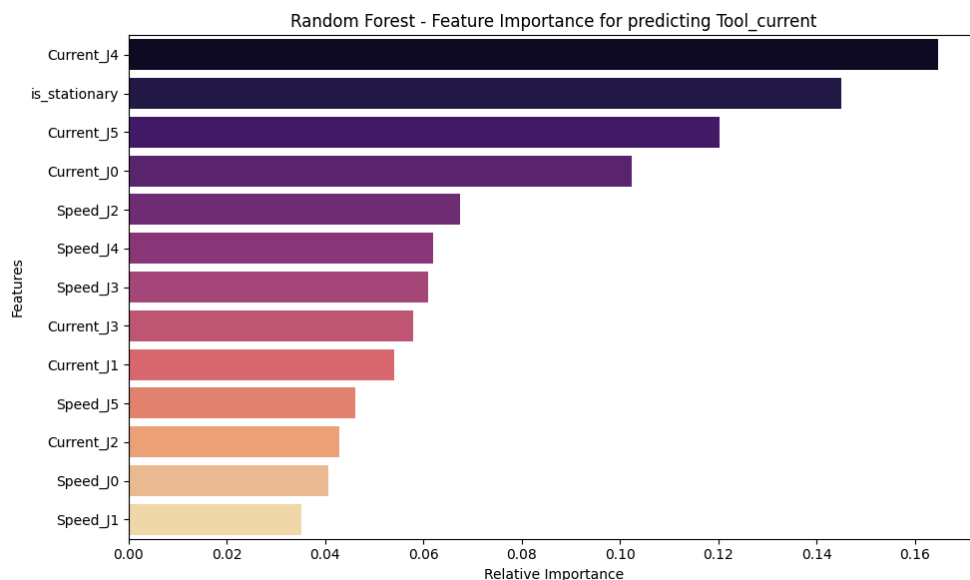
도메인 지식을 반영한 피처 엔지니어링을 통해 R²를 0.10 이상으로, 과적합 Gap을 줄이며 개선할 수 있었다.

랜덤 포레스트

단일 의사결정나무의 한계인 단순한 격자형 경계면 분할 문제를 다수의 트리 앙상블로 보완하고자 랜덤 포레스트 모델을 도입한다.

조건	Test R ²	성능 오차 (Gap)	Test MSE
파생 변수 미도입 (랜덤 포레스트 최고)	0.5604	0.2471	0.002414
파생 변수 도입 (랜덤 포레스트 최고)	0.5848	0.2363	0.002280
파생 변수 도입 + 오차 방어	0.5387	0.1476	0.002534

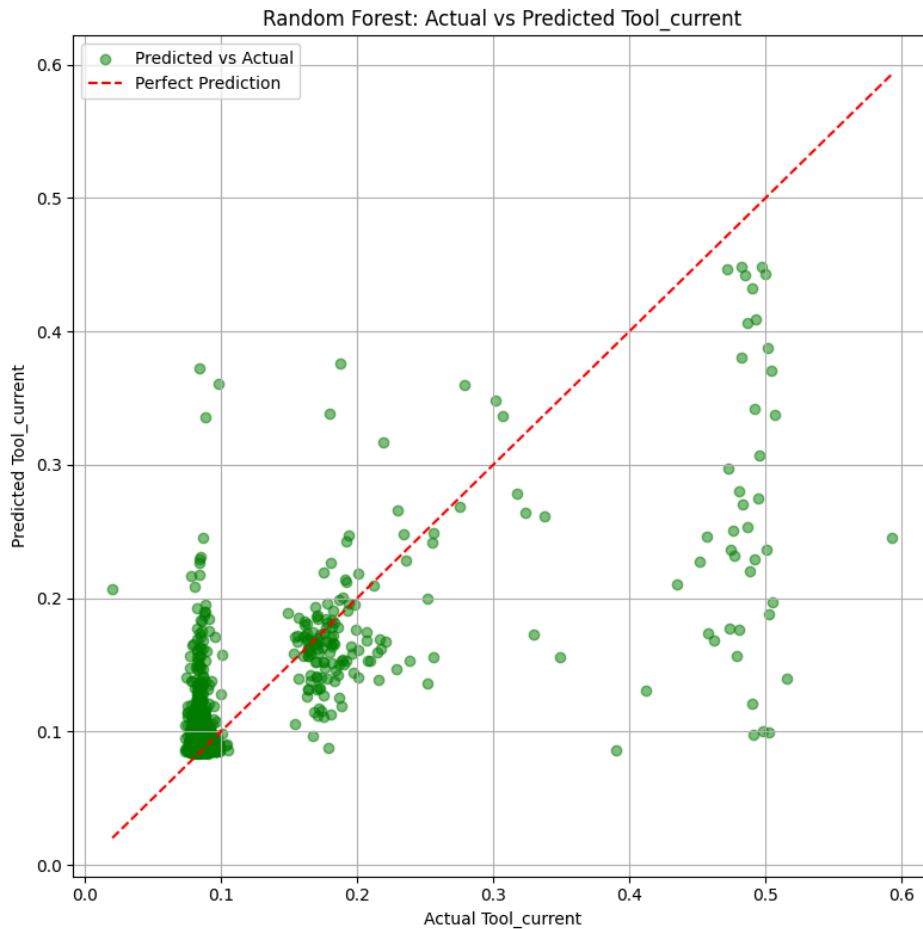
의사결정나무의 단점을 보완하기 위해 앙상블 기법인 랜덤 포레스트를 도입한 결과, 예측 성능이 더욱 상승했다. 전 단계에서 고안된 파생 변수(is_stationary)와 앙상블 모델이 결합하자, 선형 모델(R² 0.23)과 단일 트리(R² 0.46)의 한계를 넘어 최고 Test R² 0.5848을 달성하며 본 회귀 분석에서 가장 좋은 성능을 기록했다.



Current J4, is_stationary, Current J5 순으로 피처 기여도가 높다. 단일 나무 모델에서 포착되지 않던 J0 관절의 기여가 앙상블 모델을 통해 명확히 규명된다.

피쳐 기여도 분석 결과, 말단 그리퍼와 물리적으로 인접한 손목 관절(J4, J5)의 전류가 각각 1위와 3위를 차지하고, 파생 변수인 `is_stationary` 가 2위로 나타난다. 단일 트리에서 부각되지 않던 베이스 관절 J0의 기여도가 랜덤 포레스트에서 상승한 것은, 다양한 트리 조합을 통해 데이터 관계성을 분석한 결과 J0에 작용하는 모멘트 하중 역시 그리퍼의 파지 상태를 유추하는 유의미한 정보임이 입증된 결과이다.

랜덤 포레스트 모델의 근본적 한계



실제 구동에서 관찰되지 않는 중간 전류 대역인 0.25A~0.45A 영역에 예측값들이 넓게 분포하며, 실제 0.49A 부하의 데이터가 하향 예측되는 물리적 한계를 보여준다.

학습 성능 향상에도 불구하고 실제값 대비 예측값 산점도를 확인하면 다른 모델들과 마찬가지로 0.49A 대역의 파지 상태 데이터가 대부분 낮은 값으로 하향 예측된다. 이는 학습 데이터의 80% 이상을 점유하는 0.08A의 대기 상태 데이터에 모델이 편향되어, 고부하 데이터 입력 시 일부 트리가 과소 예측하여 평균치를 낮추기 때문이다.

보다 근본적으로 Tool Current의 **0.25A ~ 0.45A 대역은 로봇 구동 시 실제 전류 데이터가 희박한 물리적 공백 영역이다**. 그럼에도 불구하고 회귀 알고리즘은 두 상태 사이의 오차를 최소화하는 평균선을 도출하여 기계적으로 존재할 수 없는 중간 전류값을 예측한다. 이 분포적 왜곡이 R^2 가 0.6 수준에 머무는 근본적인 물리적 장애 요인이다.

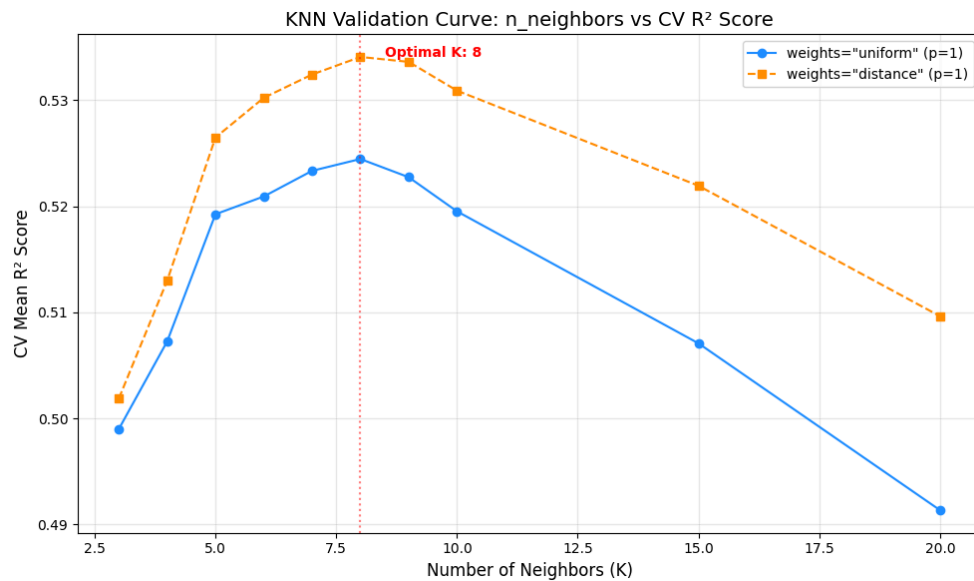
3.4 KNN

매개변수적 함수나 분기 규칙을 생성하는 모델들과 대조적으로, 별도의 최적화 학습 없이 주변 K개의 이웃 거리 정보만을 참조하는 KNN 모델을 적용하여 비교 가치를 확인한다.

조건	Test R^2	Test MSE	비고
KNN (가중치 적용, K=8)	0.5219	0.002626	학습과 평가 R^2 성능 편차 0.001로 우수한 일반화
KNN (동일 가중치, K=8)	0.5022	—	과적합 경향 제어

이웃 거리에 가중치를 부여하는 방식이 균등 가중치 적용 방식보다 일관되게 높은 성능을 나타낸다. 균등 방식은 이웃 샘플들의 평균치로 예측을 종결하므로 앞선 모델들과 유사한 '중간값 예측 오류'를 범하지만, 거리 가중 방식은 인접한 실제 데이터 분포선에 강하게 밀착하기 때문이다.

최적의 이웃 수인 $K=8$ 설정은 다음과 같은 공학적 의미를 가진다. K 가 8 미만일 경우 센서 노이즈에 과도하게 민감해져 과적합을 일으키며, K 가 8을 초과하면 다수 클래스인 0.08A 대기 데이터가 이웃으로 과도하게 편입되어 고부하 예측 강도를 하락시킨다. 이는 센서 노이즈의 임의 변동을 상쇄하면서도 상이한 운전 상태의 간섭을 방지할 수 있는 이상적인 임계 영역이 8개 내외임을 실증한다.

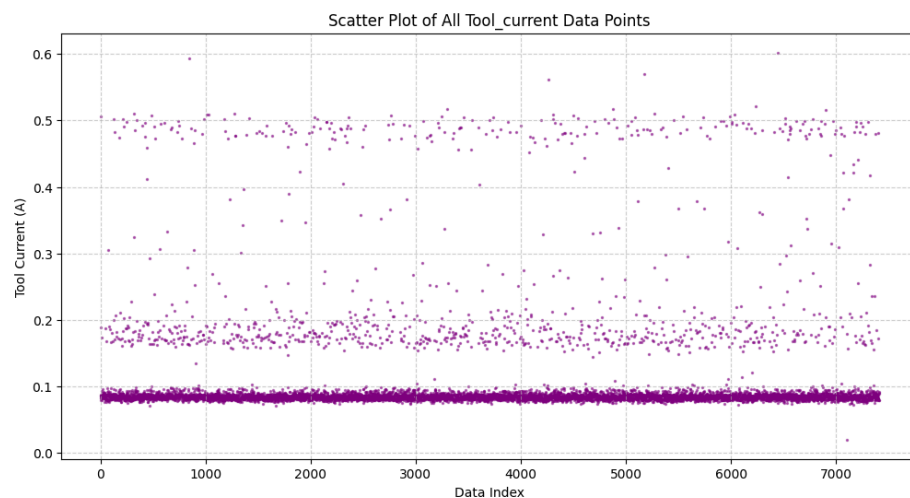


거리 가중 방식이 균등 방식 대비 전 구간에서 우세한 결정계수를 유지하며, $K=8$ 에서 최적의 임계 성능을 달성한다.

3.5 Regression 결론 및 시사점

모델별 성능 요약

모델	Test R^2	Test MSE	핵심 특성
Linear Regression (다항 피처 + Ridge)	0.2334	0.0042	다중공선성 하에서 안정적이나 비선형성 학습 불가
Decision Tree (+ <code>is_stationary</code>)	0.4697	0.0029	파생 변수로 큰 폭 개선, 단일 트리의 단순한 경계 한계
Random Forest (+ <code>is_stationary</code>)	0.5848	0.0023	최고 성능. 앙상블로 경계면 정교화
KNN (거리 가중, $K=8$)	0.5219	0.0026	학습 과정 없이 이웃 참조만으로 RF에 준하는 성능 확보



0.08A 대기, 0.17A 전이, 0.49A 파지의 3개 밴드로 명확히 나뉘며, 0.2A~0.4A 대역은 물리적 데이터가 거의 존재하지 않는 공백 영역이다.

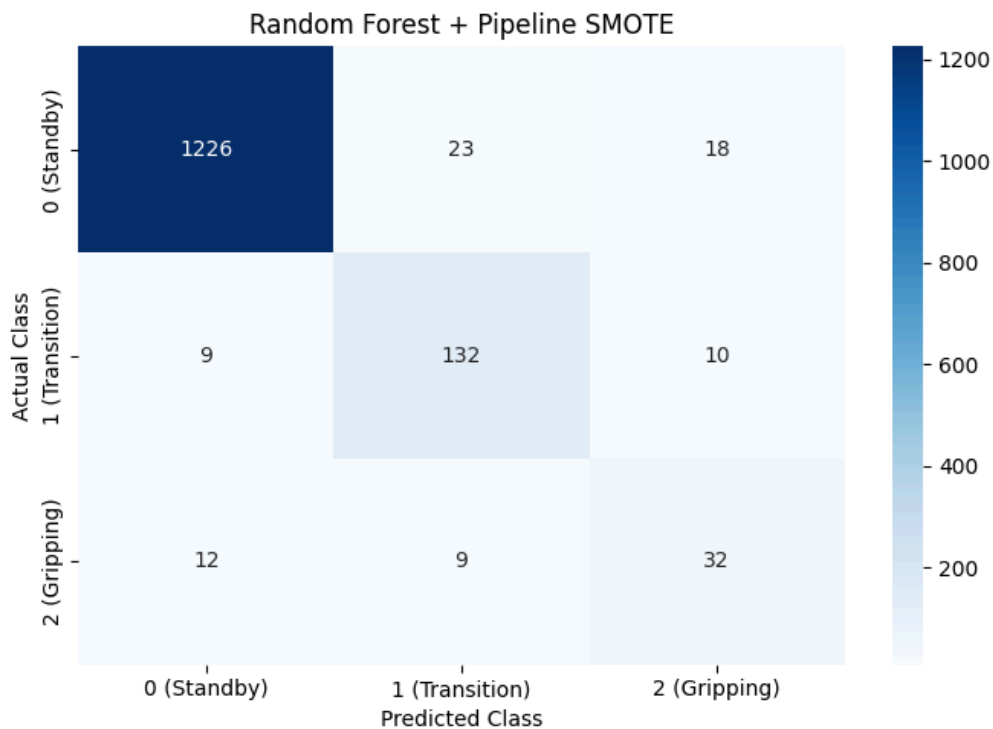
결정계수 0.6의 물리적 한계 원인

수행한 모든 회귀 분석이 R^2 0.6의 한계선을 보인 원인은 다음과 같다.

- 삼봉형 기저 분포:** 타겟 변수가 세 개의 이산적인 전력 대역에 종속되어 존재한다. 연속형 회귀 알고리즘은 오차 제곱합을 취합하여 중간 평행선을 지나도록 훈련되므로, 실제 구동 시 존재하지 않는 영역을 예측하는 근본적 물리 오류를 초래한다.
- 동일 입력 대비 다중 정답 매핑:** 로봇 관절의 각도와 속도 조합이 완전히 동일하더라도, 그리퍼의 파지 상태 여부에 따라 그리퍼 전류는 대기(0.08A) 또는 홀딩(0.49A)의 두 정답을 동시에 지닌다. 관절의 내부 센서 신호만으로는 말단 그리퍼의 물건 유무를 완벽히 결정할 수 없다는 동역학적 정보 유실이 성능의 물리적 상한을 결정한다.

분류로 관점 전환

Accuracy: 0.9449354180829368				
Macro F1: 0.7932683420380635				
	precision	recall	f1-score	support
0 (Standby)	0.98	0.97	0.98	1267
1 (Transition)	0.80	0.87	0.84	151
2 (Gripping)	0.53	0.60	0.57	53
accuracy			0.94	1471
macro avg	0.77	0.82	0.79	1471
weighted avg	0.95	0.94	0.95	1471



앞선 회귀 모델(RF)이 존재하지 않는 중간 전류값을 예측하는 물리적 오류를 범했던 한계를 극복하고자, 그리퍼 전류 데이터를 3개의 삼봉 동작 상태(0, 1, 2)로 라벨링하여 Random Forest 분류 모델을 적용해 보았다.

결과는 다음과 같다.

- Macro F1 Score 향상 (0.79): 데이터의 86%(1,267개)가 대기(Standby) 상태에 몰려있는 극심한 클래스 불균형 환경이므로 Accuracy(0.94)는 배제하였다. 클래스별 성능을 균형 있게 평가하는 Macro F1 Score가 0.79를 기록하며, 기존 회귀 모델(R^2 0.58)에 비해 로봇의 실제 상태를 훨씬 정확하게 추론해 냈다.
- 상태별 탐지력 분석:

- Standby(대기) & Transition(전이): 각각 F1-Score 0.98, 0.84를 기록하며 기계적 거동을 거의 완벽하게 식별해 냈다. 회귀 모델이 두 상태 사이에서 갈 길을 잃었던 문제가 완전히 해결되었다.
- Gripping(파지): 데이터 수가 단 53개(전체의 약 3.6%)에 불과한 극한의 희소 데이터임에도 불구하고 Recall 0.60을 기록했다. 즉, 로봇이 실제로 물건을 강하게 쥐는 핵심 순간의 60%를 성공적으로 포착해 냈다.

핵심 인사이트

비록 연속형 수치를 예측하는 회귀 문제로서는 성능의 벽이 뚜렷했으나, 로봇 관절의 간접 센서 정보만으로 랜덤포레스트 회귀 분석을 통해 그리퍼 작동 전류 분산의 **약 60%를 설명하는 데 성공한다**. 이는 모터 제어 신호와 말단 작동기 간에 유의미한 동역학적 연계 패턴이 성립함을 입증한 결과이다.

선형 모델에서 비선형 트리 및 인접 거리 기반 모델로 전환하면서 예측 성능이 **2배 이상 향상된 것은**, 로봇의 구동 로그가 단순 상관성이 아닌 **구동 조건에 따라 비선형적으로 거동 상태가 전환되는 물리 특성**을 내포하고 있음을 통계적으로 규명한 성과이다.

궁극적으로 그리퍼의 구동 상태는 연속형 수치 예측이 아닌, 물리적으로 이산화된 상태를 판단하는 분류 문제로 접근해야 타당하다는 인사이트를 도출한다. 이에 근거하여 타겟 변수를 대기, 전이, 파지 모드로 구분한 이진 및 다중 분류 모델 검증으로 태스크 패러다임을 유기적으로 전환한다.

아울러 회귀 분석을 통해 얻은 **센서 데이터의 고도 비선형성과 이산 거동 특성**이라는 교훈은, 후속 Task인 Classification에서 선형 분류기 대신 랜덤 포레스트와 같은 비선형 앙상블 알고리즘에 집중하여 연구를 수행하는 명확한 공학적 이정표가 된다.

자세한 Regression 과정은 아래의 보고서에 담겨있다.

https://github.com/kzming2007/UR3_Cobot_ML/blob/main/reports/Data_Regression_Task_Report.pdf

4. Classification — “로봇 고장을 예측할 수 있는가?”

4.1 태스크 정의 및 타당성

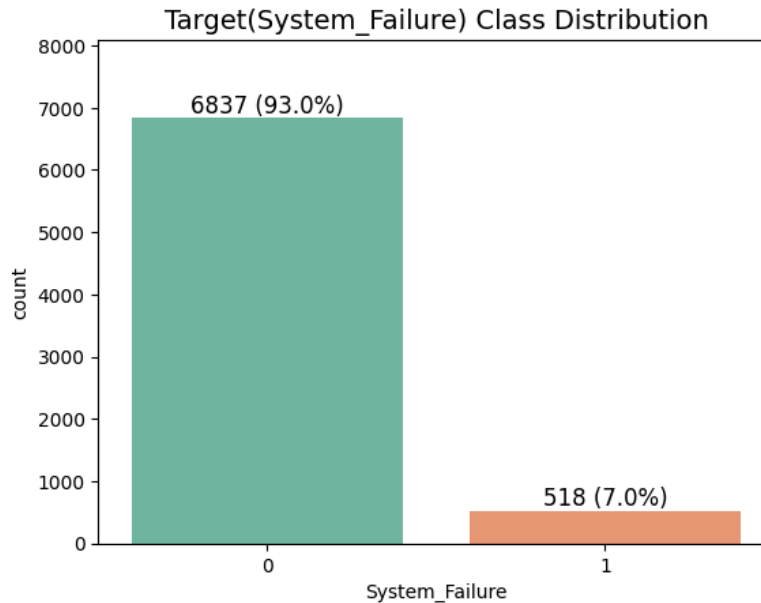
본 분류 분석의 목표는 UR3 협동로봇의 6개 관절에서 수집되는 전류, 속도, 온도 등의 센서 데이터를 활용하여 **로봇의 정상 구동 상태와 비정상 고장 상태를 정밀하게 분류**하는 최적의 머신러닝 모델을 도출하는 것이다.

타겟 변수: System Failure

로봇 팔의 작동 정지를 유발하는 두 가지 핵심 이벤트인 **Robot_ProtectiveStop** (안전 보호 정지)과 **Grip_Lost** (파지 상실) 사이에는 통계적 상관관계가 거의 존재하지 않는 것으로 분석되었다. 두 예러는 독립적으로 발생하지만, 실무적으로는 모두 “로봇이 멈추는 고장”이라는 동일한 결과를 초래하므로 하나의 파생 타겟 변수 **System_Failure** 로 통합하였다.

$$\text{System_Failure} = \begin{cases} 1 & \text{if } (\text{ProtectiveStop} = 1) \vee (\text{GripLost} = 1) \\ 0 & \text{otherwise} \end{cases}$$

클래스 불균형: “Accuracy 93%의 역설”



통합된 타겟 변수의 클래스 비율은 **정상(0) 93% : 고장(1) 7%**로 극심한 불균형을 이룬다.

이 불균형 하에서는 아무것도 학습하지 않고 모든 데이터를 정상으로 찍는 더미 분류기조차 Accuracy 93%를 달성할 수 있다. 따라서 본 분석에서는 Accuracy 대신 **Macro F1-Score**(클래스별 F1의 산술 평균)와 **ROC-AUC**를 핵심 평가 지표로 채택한다.

데이터 누수(Data Leakage) 방지

```
# 1. Stratified K-Fold 설정
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. Imblearn Pipeline 구축 (Fold 내에서 SMOTE 적용 -> Logistic Regression 학습)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('lr', LogisticRegression(random_state=42, max_iter=1000))
])

# 3. 튜닝할 파라미터 그리드 (Pipeline 사용 시 모델명__파라미터명 형태로 지정)
param_grid = {
    'lr__C': [0.01, 0.1, 1, 10, 100],
    'lr__penalty': ['l1', 'l2'],
    'lr__solver': ['saga']
}

# 4. GridSearchCV 실행 (데이터 불균형을 고려해 scoring을 'f1_macro'로 설정)
print("--- Logistic Regression 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

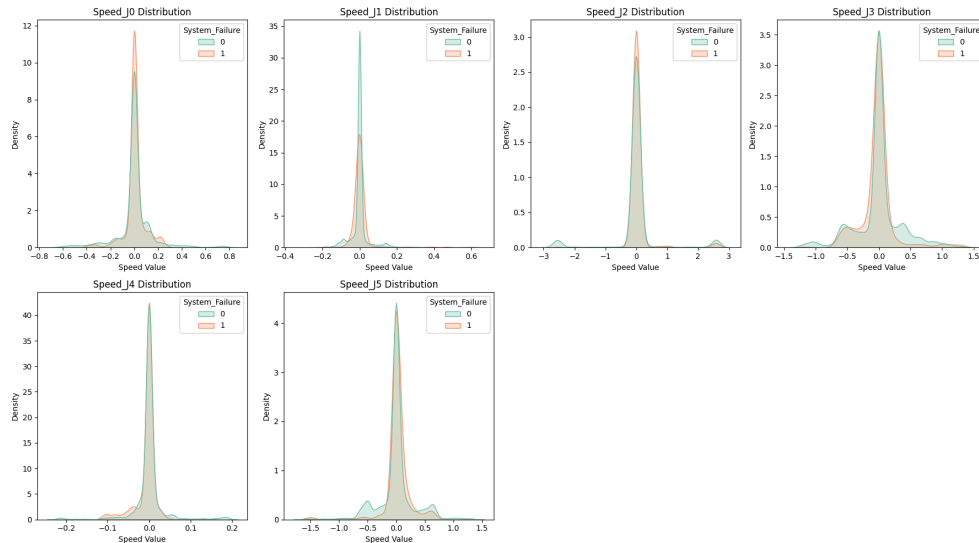
# 5. 최적 파라미터 및 결과 확인
print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    {param.replace('lr__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")
```

예시 : Logistic Regression

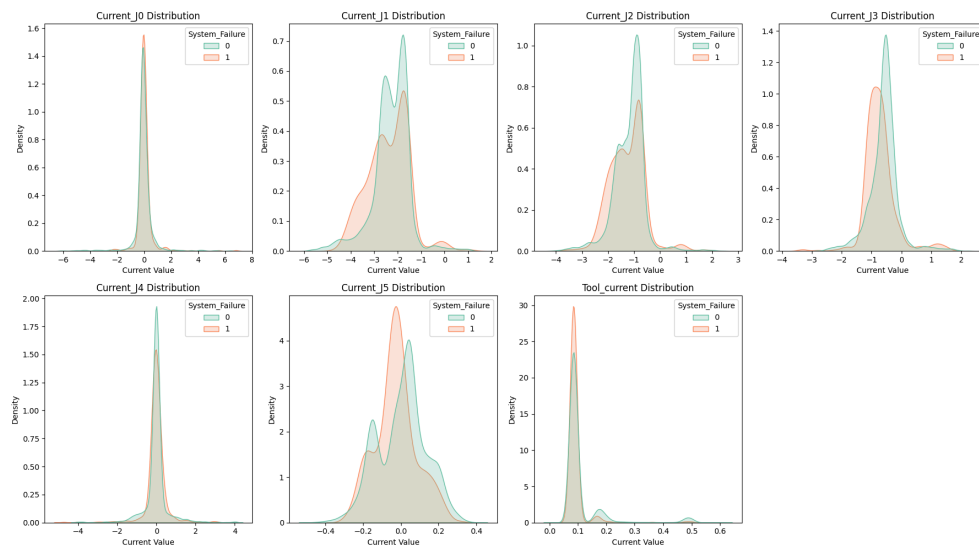
클래스 불균형을 해소하기 위해 SMOTE 오버샘플링을 적용하되, 이는 반드시 **학습(Train) 데이터에만** 적용한다. 만약 전체 데이터에 SMOTE를 적용한 뒤 Train/Test를 분할하면, SMOTE가 생성한 가상의 고장 데이터가 Test 세트에 포함되어 모델이 사실상 답을 미리 본 상태로 평가받는 데이터 누수가 발생한다. 이를 원천 차단하기 위해 교차 검증의 각 Fold 내에서만 스케일링과 오버샘플링을 수행하는 엄격한 파이프라인을 구축하였다.

4.2 파생 변수 Power 생성

EDA 단계에서 관절별 단순 센서 값의 분포를 시각화했을 때 다음과 같은 물리적 한계에 봉착한다.



- **속도 변수:** 정상 상태와 고장 상태의 데이터 분포가 거의 일치하여 단독으로는 분별력이 거의 존재하지 않는다.



- **전류 변수:** 평균값이 매우 유사하여 모델이 특정 수치 이상을 고장으로 판정하는 단순 임계값 기반의 판단을 내리기가 어렵다.

즉, 개별 센서의 정적 지표만으로는 고장의 다차원적 패턴을 포착할 수 없으므로, 로봇의 동역학적 상태를 설명하는 파생 변수 생성이 필수적이다. 이에 따라 전류와 속도의 곱을 통해 모터가 소모하는 순수 기계적 동력을 대변하는 파생 지표인 **Power**를 정의한다.

4.3 Logistic Regression

온도, 전류, 속도 및 파생 변수를 모두 활용해 학습을 진행한다. SMOTE 오버샘플링을 적용하고 하이퍼파라미터 최적화를 거쳤다.

SMOTE 단독 적용 결과

```
[선택된 1위 모델의 하이퍼파라미터]
• C: 10
• penalty: l2
• solver: saga

[Train 데이터 평가]
Accuracy (정확도) : 0.6365
F1-Macro Score   : 0.488730
ROC-AUC Score    : 0.715016

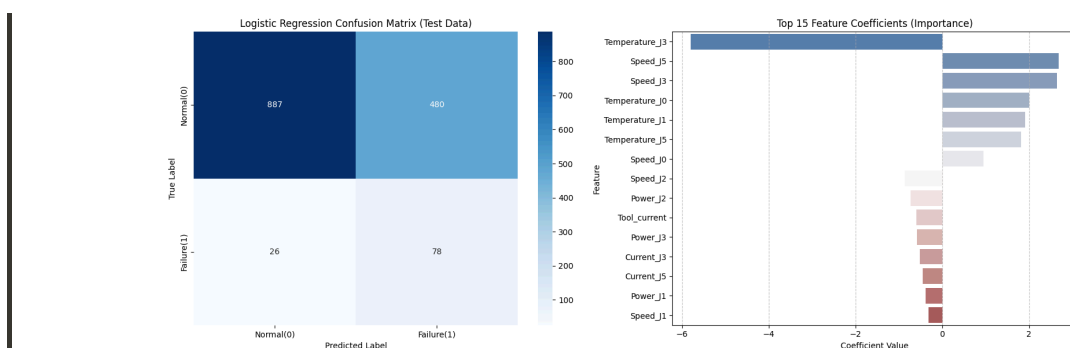
[Train 상세 분류 리포트]
|   |   |   | precision | recall | f1-score | support |
|---|---|---|---|---|---|---|
| 0 (Normal) | 0.97 | 0.63 | 0.76 | 5470 |
| 1 (Failure) | 0.13 | 0.70 | 0.21 | 414 |
| accuracy | 0.64 | 0.64 | 0.64 | 5884 |
| macro avg | 0.55 | 0.67 | 0.49 | 5884 |
| weighted avg | 0.91 | 0.64 | 0.72 | 5884 |

[Test 데이터 평가]
Accuracy (정확도) : 0.6560
F1-Macro Score   : 0.506860
ROC-AUC Score    : 0.767099

[Test 상세 분류 리포트]
|   |   |   | precision | recall | f1-score | support |
|---|---|---|---|---|---|---|
| 0 (Normal) | 0.97 | 0.65 | 0.78 | 1367 |
| 1 (Failure) | 0.14 | 0.75 | 0.24 | 104 |
| accuracy | 0.66 | 0.66 | 0.66 | 1471 |
| macro avg | 0.56 | 0.70 | 0.51 | 1471 |
| weighted avg | 0.91 | 0.66 | 0.74 | 1471 |
```

지표	정상 (0)	고장 (1)
Precision	0.97	0.14
Recall	0.65	0.75
F1-Score	0.78	0.24

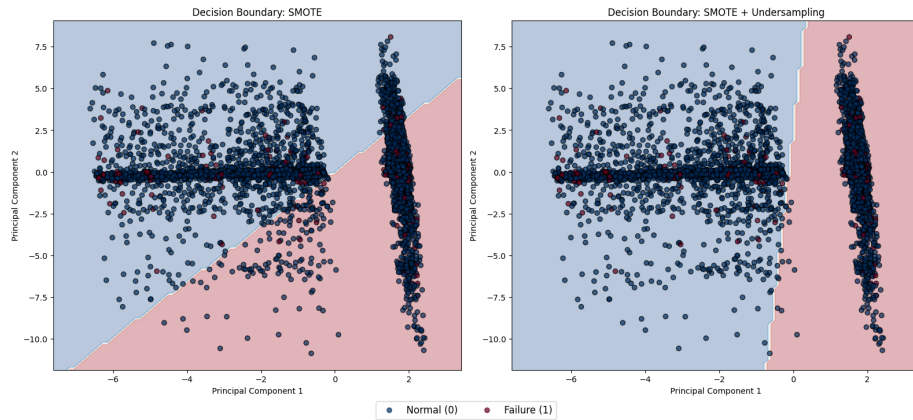
- **F1-Macro Score: 0.51 / Accuracy: 0.66**



- 오차 행렬: TN: 887 | FP: 480 | TP: 78 | FN: 26

SMOTE + UnderSampling 결합 적용

UnderSampling을 적용하여 경계 영역의 노이즈 데이터를 제거하고 경계를 정돈하려 했으나, 데이터 자체가 꼬여있는 비선형 구조이기 때문에 선형 모델(LR)의 근본적 한계를 극복하지 못했다.



두 그래프 모두 빨간 점(고장)들이 정상 데이터 무리 속에 혼재하고 있어, 선형 분리만으로는 정확한 분류가 불가능함을 시각적으로 뒷받침한다.

성능 및 오작동 메커니즘 분석

- **선형 평면의 공간 분할 실패:** 로지스틱 회귀는 26차원의 입력 공간에 단 하나의 결정 초평면(Hyperplane)에 의존하여 공간을 분할한다. 그러나 로봇의 정상과 고장 상태는 서로 중첩되어 꼬여 있는 비선형 형태를 띤다.
- **오탐지 폭증의 원인:** SMOTE로 소수 클래스를 억지로 늘린 상태에서 선형 구획을 적용하다 보니, 모델은 고장 클래스를 최대한 많이 포착하기 위해 결정 경계를 정상 영역 내부로 깊숙이 밀어 넣는다. 결과적으로 조금만 이상 징후가 보여도 무조건 고장으로 분류하여, 멀쩡한 정상 데이터 중 무려 **480건(35.1%)**을 고장으로 오판(FP)하는 한계를 보인다.

[핵심 결론] 로지스틱 회귀의 실패는 선형으로는 이 데이터를 나눌 수 없다는 결정적 증거이다. Regression에서 확인한 데이터의 비선형 특성이 분류에서도 그대로 재확인된 셈이다.

4.4 Support Vector Machine

선형 분류의 한계를 극복하기 위해 데이터를 고차원으로 매핑하여 비선형 분류가 가능한 **RBF 커널 SVM**을 적용한다.

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축
# probability=True: ROC-AUC 계산 및 확률 출력을 위해 필요 (단, 학습 시간이 조금 늘어남)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('svc', SVC(random_state=42, probability=True))
])

# 파라미터 그리드 (비선형 rbf 커널과 선형 linear 커널 비교)
param_grid = {
    'svc_C': [0.1, 1, 10],          # 규제 강도 (오차 허용 범위)
    'svc_gamma': ['scale', 'auto', 0.1], # 결정 경계의 곡률 (rbf 커널용)
    'svc_kernel': ['rbf', 'linear']    # 커널 종류
}

print("--- SVM 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 (시간이 다소 소요됩니다) ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"  {param.replace('svc_', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")
```

성능 결과 (SMOTE 적용 기준)

```

[선택된 3위 모델의 하이퍼파라미터]
• C: 1
• gamma: 0.1
• kernel: rbf

[Train 데이터 평가]
Accuracy (정확도) : 0.9016
F1-Macro Score   : 0.749483
ROC-AUC Score    : 0.932674

[Train 상세 분류 리포트]

```

	precision	recall	f1-score	support
0 (Normal)	0.99	0.90	0.94	5470
1 (Failure)	0.41	0.87	0.55	414
accuracy			0.90	5884
macro avg	0.70	0.89	0.75	5884
weighted avg	0.95	0.90	0.92	5884

```

[Test 데이터 평가]
Accuracy (정확도) : 0.8994
F1-Macro Score   : 0.728527
ROC-AUC Score    : 0.860883

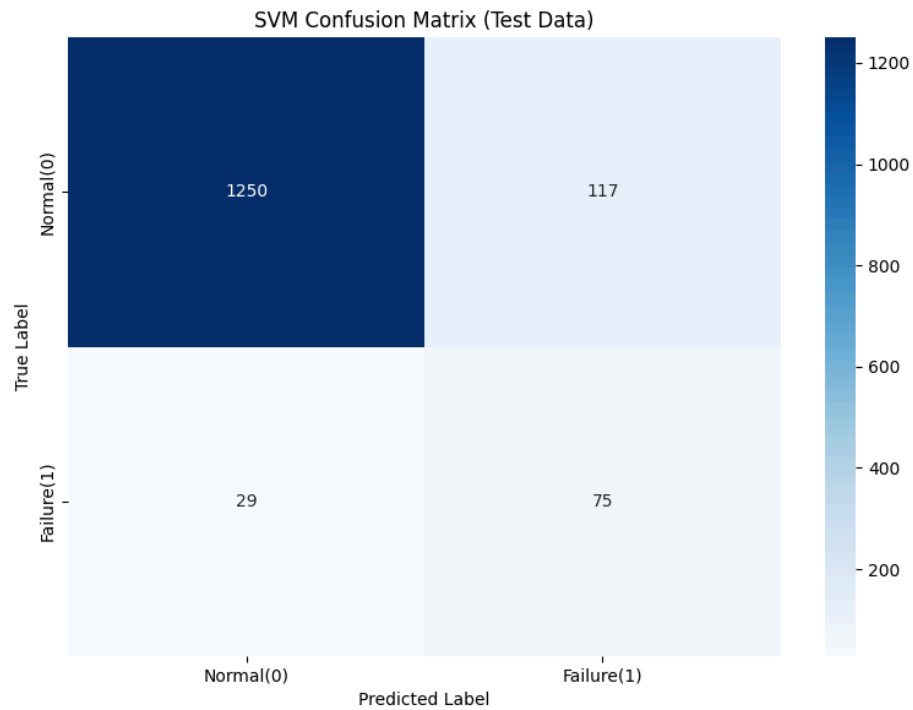
[Test 상세 분류 리포트]

```

	precision	recall	f1-score	support
0 (Normal)	0.98	0.91	0.94	1367
1 (Failure)	0.39	0.75	0.51	104
accuracy			0.90	1471
macro avg	0.68	0.83	0.73	1471
weighted avg	0.94	0.90	0.91	1471

지표	정상 (0)	고장 (1)
Precision	0.98	0.39
Recall	0.91	0.75
F1-Score	0.94	0.51

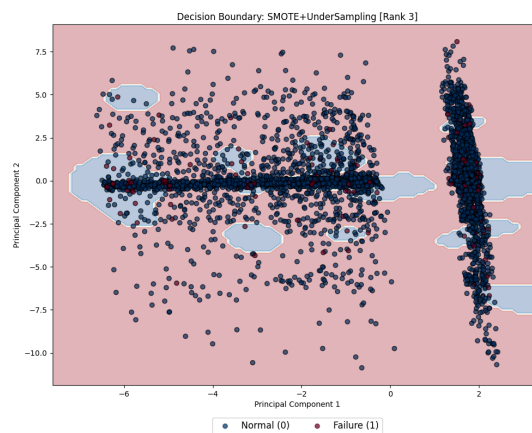
- **F1-Macro Score: 0.73** / Accuracy: 0.90 / ROC-AUC: 0.86

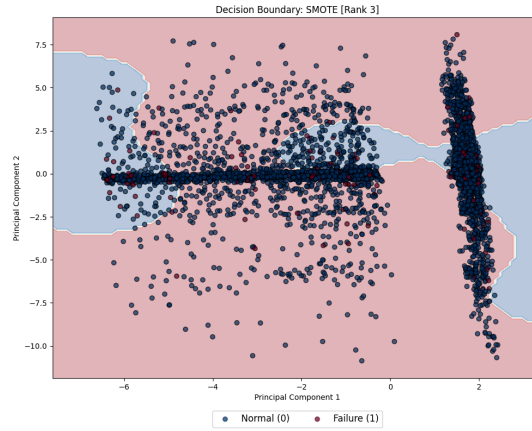


- 오차 행렬: TN: 1,250 | FP: 117 | TP: 75 | FN: 29

성능 및 경계면 형성 분석

- 결정 경계의 유연화: RBF 커널은 데이터를 고차원 공간으로 매핑하여 직선이 아닌 부드럽게 감싸는 곡선형 경계 포켓을 형성한다. 정상 데이터 분포 속에 산재한 소수 고장 데이터를 감싸는 곡선형 결정 경계를 효과적으로 구축했다.
- 오탐 제어의 대폭 개선: 경계가 곡선 형태로 형성되면서 정상 영역을 침범하지 않게 되어, 선형 모델의 치명적 약점이었던 오탐 (FP)을 480건 → 117건으로 75.6% 격감시켰다.
- SMOTE + ENN 결합 효과: ENN으로 경계선의 노이즈를 추가 제거한 결과, 오탐이 115건으로 소폭 추가 감소하여 일반화 성능에 긍정적 영향을 미쳤다.





[핵심 결론] 비선형 커널로 구부리면 해결된다는 가설이 증명되었다. LR 대비 F1-Macro가 0.51 → 0.73으로 상승하며, 비선형 모델의 필요성이 확정되었다.

4.5 Decision Tree

직관적인 의사결정 규칙을 통해 로봇 제어 규칙을 해석할 수 있도록 단일 의사결정나무 모델을 학습한다.

```
print("\n" + "-"*50)
target_rank = 2 # 1위 모델 상세 분석
print(f"--- [상세 분석] (target_rank)위 모델 최종 평가지표 ---")
print("-"*50)

selected_row = top_models[top_models['rank_test_score'] == target_rank].iloc[0]
selected_params = selected_row['params']

print(f"\n[선택된 (target_rank)위 모델의 하이퍼파라미터]")
for param, value in selected_params.items():
    print(f"  • {param.replace('dt_', '')}: {value}")

# 재학습 모델 정의
selected_model = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('dt', DecisionTreeClassifier(
        criterion=selected_params.get('dt_criterion', 'gini'),
        max_depth=selected_params.get('dt_max_depth', None),
        min_samples_leaf=selected_params.get('dt_min_samples_leaf', 1),
        min_samples_split=selected_params.get('dt_min_samples_split', 2),
        random_state=42, class_weight='balanced'
    ))
])
selected_model.fit(X_train, y_train)

# 예측 클래스 및 확률 계산
y_train_pred_selected = selected_model.predict(X_train)
y_train_prob_selected = selected_model.predict_proba(X_train)[:, 1]

y_test_pred_selected = selected_model.predict(X_test)
y_test_prob_selected = selected_model.predict_proba(X_test)[:, 1]

# 평가지표 출력 함수
def print_metrics(y_true, y_pred, y_prob, dataset_name):
    acc = accuracy_score(y_true, y_pred)
    f1_macro = f1_score(y_true, y_pred, average='macro')
    roc_auc = roc_auc_score(y_true, y_prob)

    print(f"\n[{dataset_name}] 데이터 평가")
    print(f"Accuracy (정확도) : {acc:.4f}")
    print(f"F1-Macro Score : {f1_macro:.6f}")
    print(f"ROC-AUC Score : {roc_auc:.6f}")
    print(f"\n[{dataset_name}] 상세 분류 리포트")
    print(classification_report(y_true, y_pred, target_names=["0 (Normal)", "1 (Failure)"]))
```

성능 결과


```

[선택된 2위 모델의 하이퍼파라미터]
• criterion: entropy
• max_depth: 15
• min_samples_leaf: 7
• min_samples_split: 5

[Train 데이터 평가]
Accuracy (정확도) : 0.9514
F1-Macro Score   : 0.849432
ROC-AUC Score    : 0.988685

[Train 상세 분류 리포트]

```

	precision	recall	f1-score	support
0 (Normal)	0.99	0.95	0.97	5470
1 (Failure)	0.60	0.91	0.73	414
accuracy			0.95	5884
macro avg	0.80	0.93	0.85	5884
weighted avg	0.97	0.95	0.96	5884

```

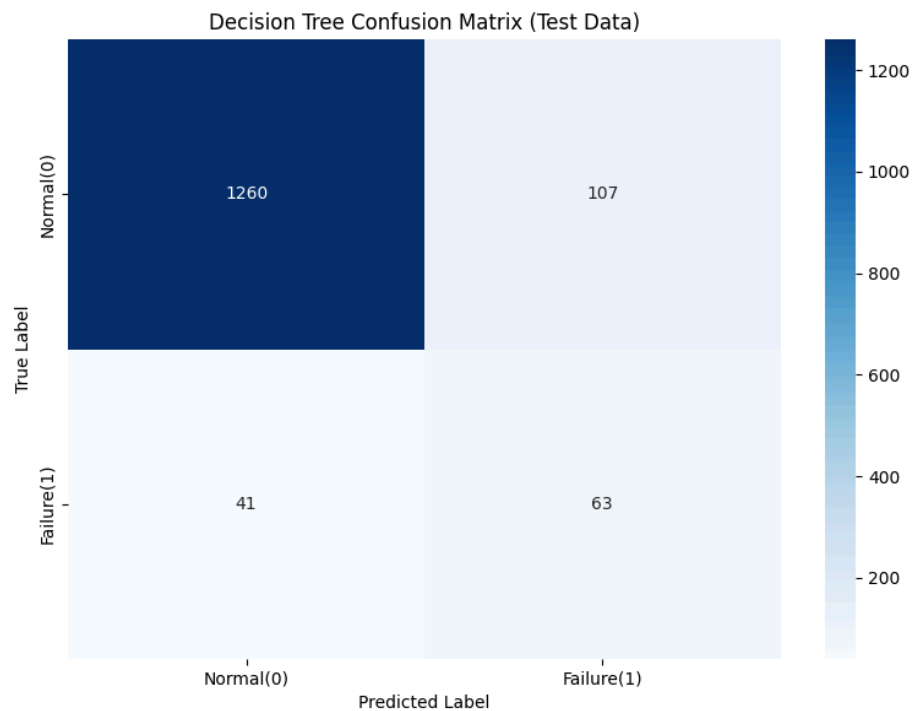
[Test 데이터 평가]
Accuracy (정확도) : 0.9001
F1-Macro Score    : 0.707077
ROC-AUC Score     : 0.821581

[Test 상세 분류 리포트]

```

	precision	recall	f1-score	support
0 (Normal)	0.97	0.92	0.94	1367
1 (Failure)	0.38	0.62	0.47	104
accuracy			0.90	1471
macro avg	0.67	0.77	0.71	1471
weighted avg	0.93	0.90	0.91	1471

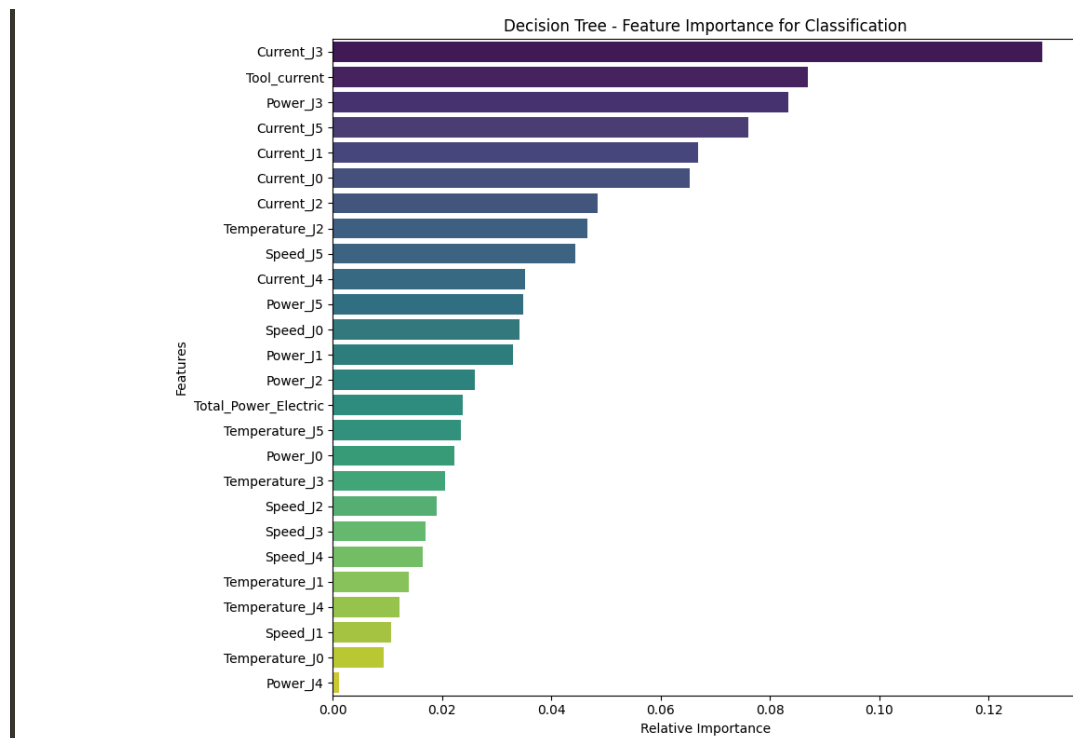
- **F1-Macro Score: 0.71** / Accuracy: 0.90



- 오차 행렬: TN: 1,260 | FP: 107 | TP: 41 | FN: 63

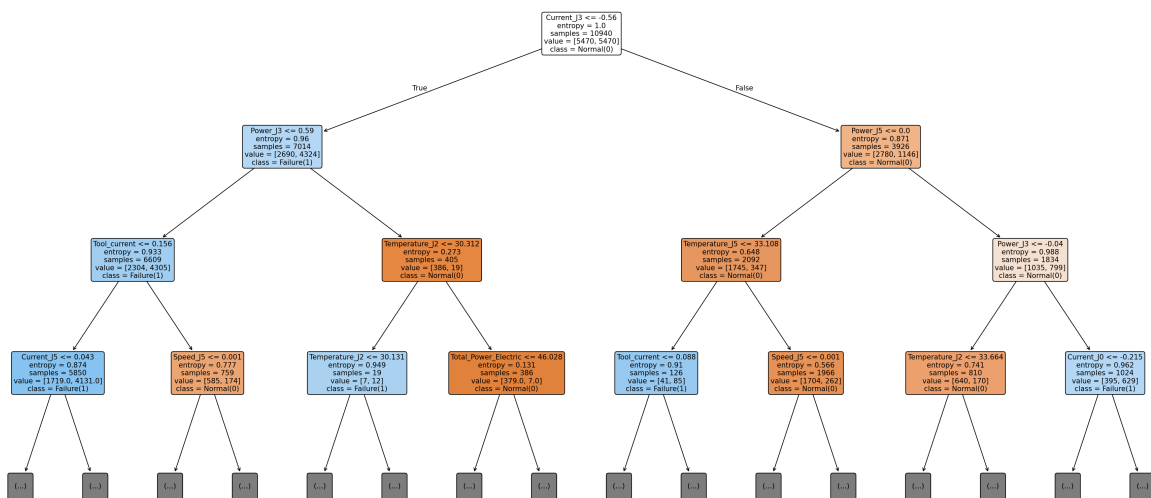
수치적 성능만 보면 SVM(F1-Macro 0.73)에 미치지 못한다. 그러나 의사결정나무의 진정한 가치는 성능 수치가 아닌, **모델 해석 가능성(Interpretability)**에 있다.

피처 중요도에서 전류 변수의 부상



앞선 모델들(LR, SVM)까지는 온도, 속도를 포함한 전체 26개 피처를 균등하게 투입했다. 그러나 의사결정나무의 피처 중요도를 확인한 결과, **온도·속도를 중시하던 기존 관점에서 '전류(Current)' 변수가 고장 예측의 핵심으로 부상하기** 시작했다.

Decision Tree Structure Analysis (Top 3 Levels)



특히, 트리 시각화 결과 **Current J3(팔꿈치 인접 관절)**가 루트 노드를 차지했다. 이는 모델이 고장 여부를 판단할 때 가장 먼저 참조하는 변수가 전류임을 의미하며, 이후 Random Forest의 피처 중요도 분석에서 더욱 명확히 검증된다.

축 정렬 분할의 구조적 한계

- **계단식 경계의 물리적 부적합:** 의사결정나무는 축에 수직 또는 수평인 격자 형태로만 경계를 분할한다. 그러나 고장 예측의 핵심 변수인 Power(전력)는 속도와 전류의 곱으로 계산되어 **하이퍼볼릭 반비례 곡선**을 형성한다. 이 둥근 곡선 경계를 단일 나무가 표현하려면 계단식 블록을 촘촘하게 쌓아 근사해야 한다.
- **미탐과 오탐의 동시 유발:**
 - **미탐(FN: 41건):** 계단식 블록 바깥으로 벗어난 실제 고장 데이터를 놓침
 - **오탐(FP: 107건):** 사각형 블록 내부의 빈 공간에 포함된 정상 데이터를 고장으로 오인

4.7 Random Forest (최종 최적 앙상블 모델)

의사결정나무 200개를 배깅(Bagging) 기법으로 결합하고 무작위 피처 선택으로 변수 다양성을 극대화한 앙상블 모델이다.

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축 (SMOTE -> RandomForest)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('rf', RandomForestClassifier(random_state=42))
])

# 파라미터 그리드
param_grid = {
    'rf__n_estimators': [100, 200],          # 트리의 개수 (앙상블 규모)
    'rf__max_depth': [7, 8, 10],             # 트리의 최대 깊이
    'rf__min_samples_split': [8, 12, 14],    # 노드를 분할하기 위한 최소 샘플 수
    'rf__min_samples_leaf': [15, 20, 25, 30]  # 리프 노드가 되기 위한 최소 샘플 수
    'rf__n_estimators': [100, 200],          # 숲의 크기 (트리가 많을수록 안정적, 과적합을 유발하지 않음)
    'rf__max_depth': [15, 20, 25],           # 트리의 최대 깊이 제한
    'rf__min_samples_split': [8, 10, 15],    # 노드를 쪼개기 위한 최소 샘플 수 (허를 높이기)
    'rf__min_samples_leaf': [2, 4, 6],       # 이파리(끝 노드)에 남아야 하는 최소 샘플 수 (진가치 치기)
    'rf__max_features': ['sqrt']             # 분할 시 고려할 최대 변수 개수 (트리들의 다양성 확보)
}

print("--- Random Forest 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"  • {param.replace('rf__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")
```

성능 결과 (SMOTE 단독 적용)

```
[선택된 4위 모델의 하이퍼파라미터]
• max_depth: None
• min_samples_leaf: 4
• min_samples_split: 10
• n_estimators: 200

[Train 데이터 평가]
Accuracy (정확도) : 0.9879
F1-Macro Score   : 0.956444
ROC-AUC Score    : 0.999100

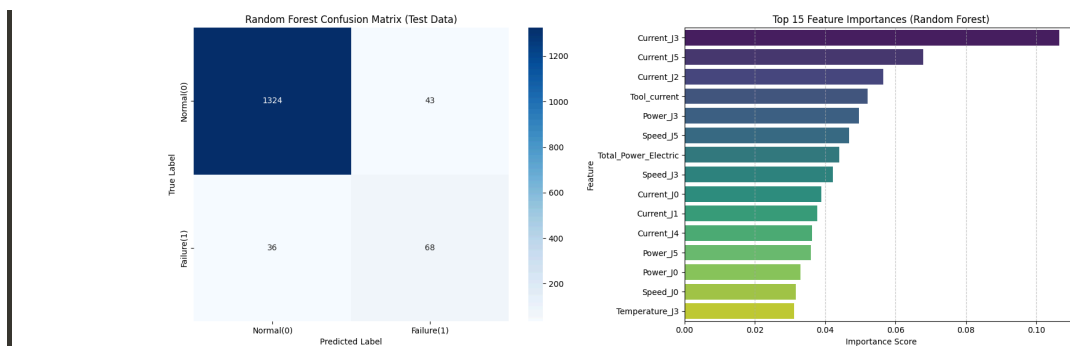
[Train 상세 분류 리포트]
| | | | precision | recall | f1-score | support |
|---|---|---|---|
0 (Normal)      | 1.00   | 0.99    | 0.99    | 5470
1 (Failure)     | 0.87   | 0.98    | 0.92    | 414
|
| accuracy      |         |         | 0.99    | 5884
| macro avg     | 0.93   | 0.98    | 0.96    | 5884
| weighted avg  | 0.99   | 0.99    | 0.99    | 5884

[Test 데이터 평가]
Accuracy (정확도) : 0.9429
F1-Macro Score    : 0.795370
ROC-AUC Score     : 0.935675

[Test 상세 분류 리포트]
| | | | precision | recall | f1-score | support |
|---|---|---|---|
0 (Normal)      | 0.97   | 0.96    | 0.97    | 1367
1 (Failure)     | 0.58   | 0.66    | 0.62    | 104
|
| accuracy      |         |         | 0.94    | 1471
| macro avg     | 0.78   | 0.81    | 0.80    | 1471
| weighted avg  | 0.95   | 0.94    | 0.94    | 1471
```

지표	정상 (0)	고장 (1)
Precision	0.97	0.58
Recall	0.96	0.66
F1-Score	0.97	0.62

- **F1-Macro Score: 0.80 / Accuracy: 94.63% / ROC-AUC: 0.94**



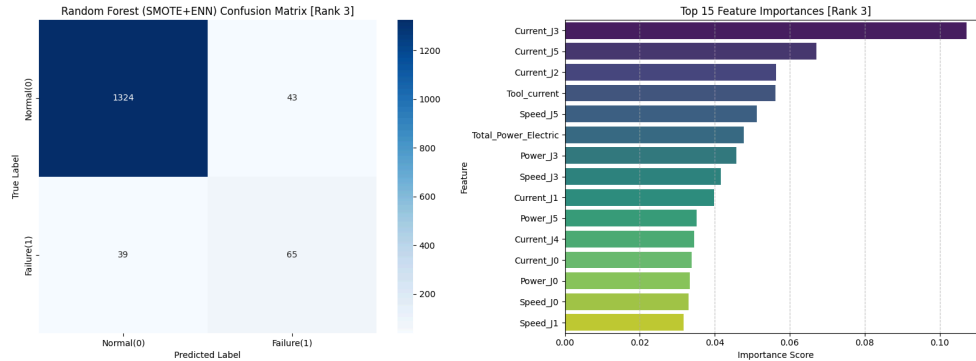
- 오차 행렬: TN: 1,324 | FP: 43 | TP: 68 | FN: 36

오탐지 91% 감소: LR 대비 최종 비교

모델	FP (오탐)	FP 비율	감소폭
Logistic Regression	480	35.1%	—
SVM (RBF)	117	8.6%	-75.6%

모델	FP (오탐)	FP 비율	감소폭
Decision Tree	107	7.8%	-77.7%
Random Forest	43	3.1%	-91.0%

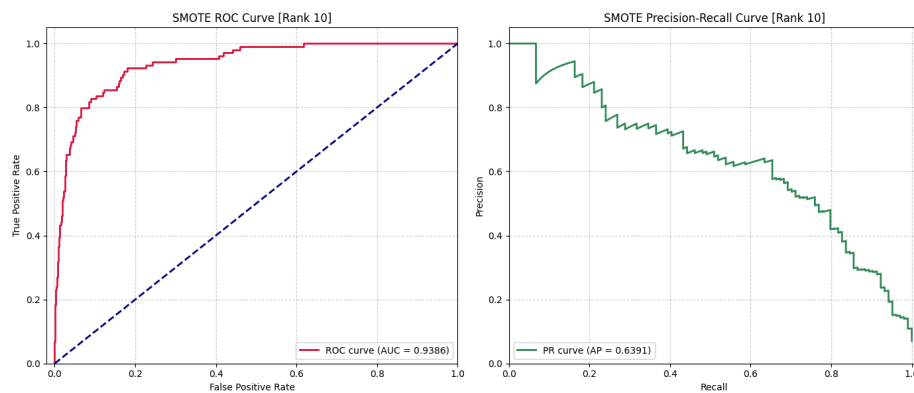
RF 내장 노이즈 방어력: SMOTE+UnderSampling 효과가 미미한 이유



다른 모델들(LR, SVM, KNN)에서는 SMOTE 단독 적용 시 경계 영역의 노이즈로 인해 성능 저하가 발생하여 ENN 결합이 필수적이었다. 그러나 **RF는 SMOTE 단독 적용과 UnderSampling 결합 적용 간의 성능 편차가 거의 없었다.**

이는 200개의 개별 트리가 다수결 투표(를 수행하는 과정에서 경계면의 노이즈 데이터에 의한 개별 트리의 오류가 다른 트리들에 의해 자연스럽게 상쇄되기 때문이다. 즉, RF는 노이즈 정제를 알고리즘 내부에서 자체적으로 수행하는 셈이다.

PR Curve 분석



PR Curve 분석 결과, RF 모델은 **Recall이 60% 수준일 때에도 Precision이 약 60%를 유지하는** 균형 잡힌 곡선을 보여주었다. 이는 고장을 더 많이 잡아내려 할 때(Recall ↑) 오탐이 폭증하던 다른 모델들과 달리, RF가 고장 탐지 강도를 높여도 정밀도가 급격히 무너지지 않음을 의미한다.

Random Forest 성능 우위 원인 분석

- 다차원 비선형 결정 경계 형성:** 개별 트리들이 생성한 수직·수평 분할 경계선들이 수백 개씩 중첩되면서, 단일 트리가 계단식으로 근사해야 했던 곡선 경계를 **모자이크 형태의 고밀도 결정 경계**로 정교하게 모델링한다.
- 내장 노이즈 저항력:** 앙상블 다수결 투표가 경계면 노이즈의 영향을 자연 상쇄하여, 별도의 ENN 정제 없이도 안정적인 성능을 확보한다.

4.6 K-Nearest Neighbors

데이터 간의 유클리디안 및 맨해튼 거리를 계산하여 이웃 간의 분류 다수결을 통해 동작하는 KNN 분류 모델을 구축한다.

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('enn', EditedNearestNeighbours(n_neighbors=3, kind_sel='all')), # ENN 추가!
    ('knn', KNeighborsClassifier())
])

param_grid = {
    'knn__n_neighbors': [3, 5, 7, 9, 11],
    'knn__weights': ['uniform', 'distance'],
    'knn__metric': ['euclidean', 'manhattan']
}

print("--- [SMOTE+ENN] KNN 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline, param_grid=param_grid, cv=skf, scoring='f1_macro', n_jobs=-1
)
grid_search.fit(X_train_scaled, y_train)

cv_results_df = pd.DataFrame(grid_search.cv_results_)
top_models = cv_results_df.sort_values('rank_test_score').head(10)

comparison_data = []
print("--- [모델 비교] 상위 10개 KNN (SMOTE+ENN) 성능 종합 ---")

for index, row in top_models.iterrows():
    params = row['params']
    rank = int(row['rank_test_score'])
    cv_f1 = row['mean_test_score']

    model = ImbPipeline([
        ('smote', SMOTE(random_state=42)),
        ('enn', EditedNearestNeighbours(n_neighbors=3, kind_sel='all')),
        ('knn', KNeighborsClassifier(
            n_neighbors=params.get('knn__n_neighbors', 5),
            weights=params.get('knn__weights', 'uniform'),
            metric=params.get('knn__metric', 'euclidean')
        ))
    ])
    model.fit(X_train_scaled, y_train)

    y_train_pred = model.predict(X_train_scaled)
    y_test_pred = model.predict(X_test_scaled)
    v_test_prob = model.predict_proba(X_test_scaled)[:, 1]

```

성능 결과 (SMOTE + ENN 적용)

```

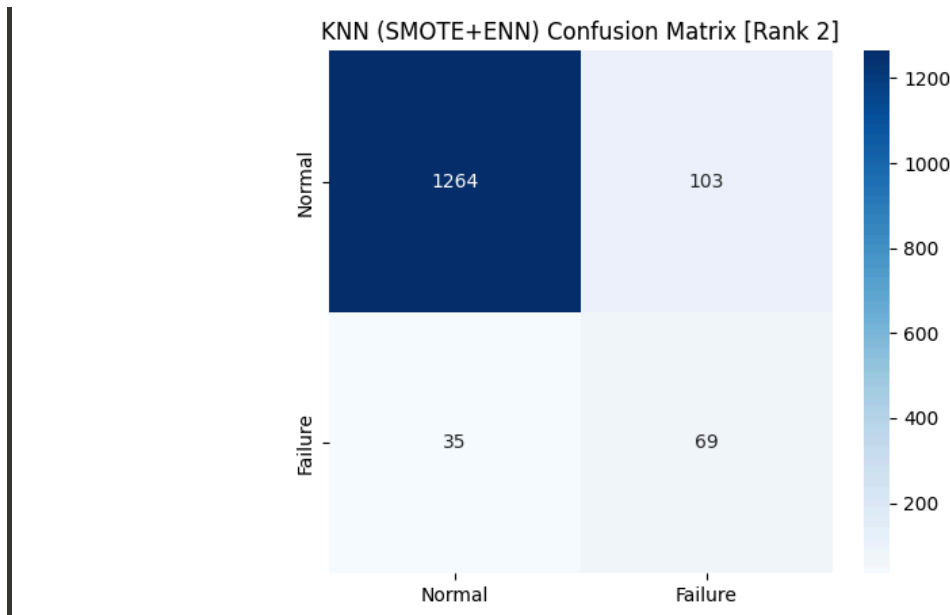
--- [상세 분석] 2위 모델 최종 평가지표 및 시각화 ---

[Test 데이터 평가]
Accuracy : 0.9062
F1-Macro : 0.724119
ROC-AUC : 0.843484

```

	precision	recall	f1-score	support
0 (Normal)	0.97	0.92	0.95	1367
1 (Failure)	0.40	0.66	0.50	104
accuracy			0.91	1471
macro avg	0.69	0.79	0.72	1471
weighted avg	0.93	0.91	0.92	1471

- **F1-Macro Score: 0.72** / Accuracy: 0.90 / ROC-AUC: 0.85



- 오차 행렬: TN: 1,264 | **FP: 103** | TP: 69 | FN: 35

차원의 저주와 거리 왜곡 분석

KNN은 RF보다 오탐(FP)이 **약 2.5배(103건 vs 43건)** 많다. 그 원인은 고차원 공간의 구조적 한계에 있다.

- 26차원 거리 왜곡:** KNN은 26개의 독립변수 거리를 모두 동일한 비중으로 계산하여 이웃을 탐색한다. 이 과정에서 상대적으로 분별력이 낮은 속도 및 온도 변수들의 미세한 거리 차이가 유클리디안 거리에 노이즈로 누적된다. 결과적으로 **고장 감지의 핵심인 전류와 Power 변수 간의 거리 정보가 희석되는** 왜곡이 발생한다.
- SMOTE 가상 이웃 오인:** 정상 영역의 빈 공간에 SMOTE가 생성한 가짜 고장 데이터 포인트들이 산재하면서, 정상 데이터 주변에 가짜 고장 이웃들이 무더기로 매칭되어 다수의 오탐을 유발한다.

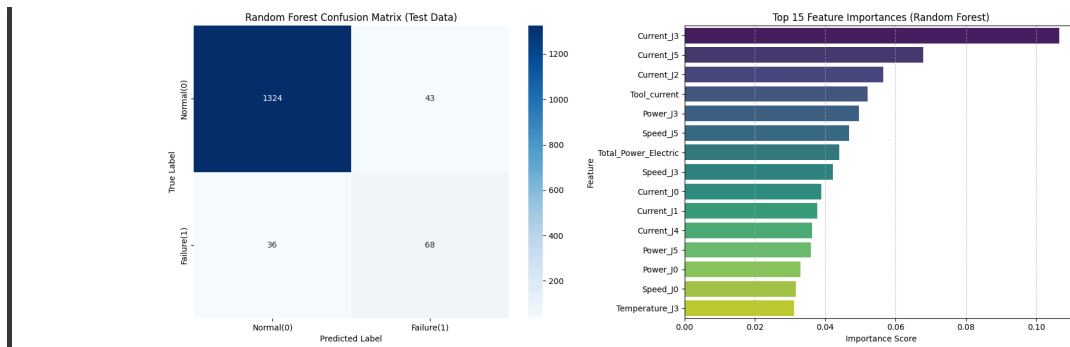
[핵심 결론] RF가 피처를 무작위로 선별하여 중요 변수에 자연스럽게 가중치를 부여하는 반면, KNN은 **26개 피처를 동일 비중으로 거리를 계산하여** 핵심 신호가 희석된다. 이것이 RF 대비 오탐이 2.5배 많은 근본 원인이다.

4.7 Classification 결론

5개 모델 성능 비교

모델	F1-Macro	FP (오탐)	ROC-AUC	핵심 특성
Logistic Regression	0.51	480	0.77	선형 경계의 근본적 실패
SVM (RBF)	0.73	117	0.86	비선형 커널로 곡선 경계 형성
Decision Tree	0.71	107	0.82	피처 중요도 해석, 축 정렬 분할 한계
KNN	0.72	103	0.85	차원의 저주로 핵심 신호 희석
Random Forest	0.80	43	0.94	양상불 경계 정교화 + 내장 노이즈 방어

핵심 발견: "전류 변수(Current J3, J5, J2)가 고장 예측의 핵심"



Random Forest의 피쳐 중요도 분석 결과, 고장 예측에 가장 크게 기여하는 상위 변수는 **Current J3, Current J5, Current J2** 순으로 전류 변수가 압도적으로 상위를 차지했다. 온도와 속도 변수는 상대적으로 낮은 기여도를 보였다.

이 결과는 Decision Tree에서 Current J3가 루트 노드를 차지한 것과 완벽히 일치하며, **로봇 고장을 감지하는 가장 강력한 신호는 관절의 전류이다**라는 핵심 발견을 도출한다.

Clustering으로의 자연스러운 전환

“전류 변수가 고장을 예측하는 핵심 신호라면, 그 전류 변수만으로 로봇의 운영 상태 자체를 나눌 수 있지 않을까?”

이 질문이 다음 태스크인 Clustering의 출발점이 된다. Classification에서 검증된 전류 변수(Group A)만을 활용하여 로봇의 거시적인 운영 상태를 비지도 학습으로 군집화하고, 그 결과가 실제 고장 패턴과 어떻게 연결되는지를 검증한다.

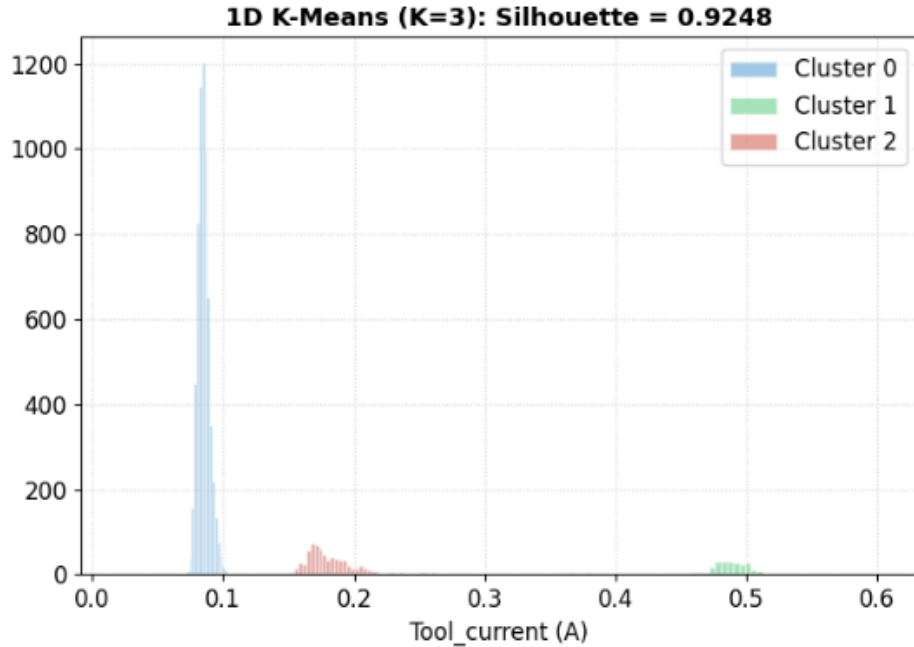
더욱 많고 자세한 Classification 과정은 아래의 보고서에 담겨있다.

https://github.com/kzming2007/UR3_Cobot_ML/blob/main/reports/Data_Classification_Task_Report.pdf

5. Clustering — “로봇은 몇 가지 상태로 움직이는가?”

정답 라벨 없이 관절 센서 데이터만을 활용하여 UR3 협동로봇의 거시적인 물리적 구동 상태를 비지도 학습으로 규명한다. K-Means, DBSCAN, 계층적 군집화 알고리즘을 다각도로 교차 실험하여 단순 통계적 정량 지표를 넘어선 동역학적 거동 상태를 도출한다.

5.1 단변량 군집화의 한계와 다차원 공간 확장



가상 센싱의 타겟인 그리퍼 전류 단일 피쳐만을 사용하여 1차원 군집화를 우선 수행한다. 이후 분석 공간을 다차원으로 확장하여 통계 수치와 물리적 유의성 간의 트레이드오프 관계를 검증한다.

- **단변량 K-Means 군집화의 한계:**

그리퍼 전류 단일 피쳐에 K-Means(K=3)를 적용한 결과, 0.9248이라는 매우 높은 실루엣 점수를 기록한다. 그러나 이는 데이터 분포의 불연속적 띠를 기준으로 경계를 단순 분할하는 임계값 필터링에 불과하다. 그리퍼 단독 신호만으로는 로봇 어깨와 팔꿈치 관절에 가해지는 중력 하중과 모멘트 부하의 영향을 전혀 설명하지 못하는 도메인적 한계를 지닌다.

- **3차원 공간 확장 실험:**

그리퍼 전류와 더불어 로봇 본체의 자중을 물리적으로 직접 지탱하는 대형 관절인 어깨 J1 및 팔꿈치 J2 관절 전류를 결합하여 3차원 공간으로 분석을 확장한다. 다차원 공간 내 센서 노이즈 유입으로 실루엣 점수는 0.4614로 하락하지만, 교차표 분석 결과 1차원 공간에서 동일 군집으로 묶였던 대기 데이터들이 어깨와 팔꿈치의 중력 보상 동역학에 따라 유기적으로 재배치되는 거동을 포착한다. 이는 수학적 지표 하락이 분석 실패가 아닌 물리적 타당성 확보 과정임을 보여준다.

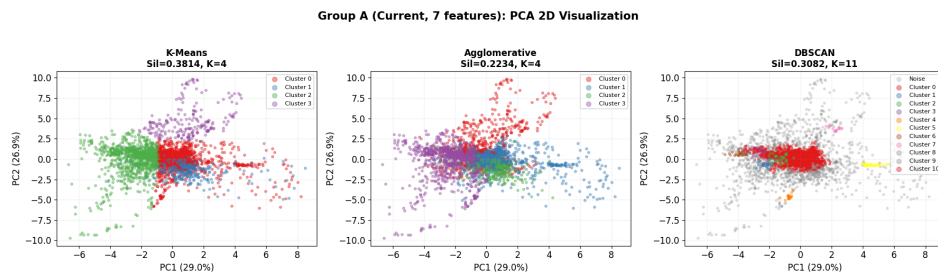
5.2 피쳐 그룹 및 군집 알고리즘 교차 비교 분석

입력 변수들을 물리적 특성에 따라 세 가지 피쳐 그룹으로 분류하고, 세 가지 군집 알고리즘을 적용하여 최적의 이상 상태 격리 성능을 보이는 조합을 탐색한다.

- **피쳐 그룹 정의:**

- **Group A:** 관절 전류 7개 변수 (전기적 하중)
- **Group B:** 관절 전류 및 속도 13개 변수 (기동 동역학)
- **Group C:** 전체 센서 19개 변수 (종합 열역학 상태)

- **군집 알고리즘별 성능 비교:**



피쳐 그룹	군집 알고리즘	설정 군집 수 (K)	실루엣 점수	군집화 적합 상태
단변량 그리퍼 전류	K-Means	K=3	0.9248	단순 구간 분할 (1D 한계)
Group A (전류 7)	K-Means	K=4	0.3814	다차원 전류 패턴 정상 식별
Group A (전류 7)	DBSCAN	K=4 (eps=2.0)	0.5338	정상 편향 밀도 장벽 오작동 발생
Group A (전류 7)	계층적	K=4	0.2234	군집별 실루엣 폭의 파편화 경향

• DBSCAN의 밀도 장벽 오작동 분석:

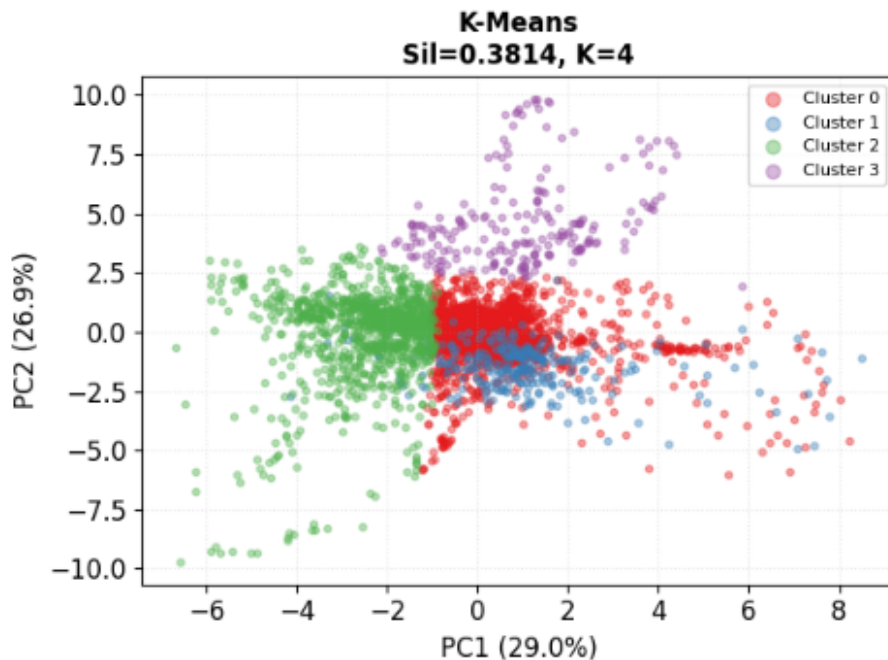
DBSCAN 알고리즘은 전체 데이터의 86% 이상이 0.08A의 대기 상태에 초고밀도로 군집화되어 있는 불균형 특성 하에서 취약성을 보인다. 대기 상태 영역의 밀도 임계치를 학습하는 과정에서 고전류 구동 구간과 에러 발생 구간을 단순히 노이즈 포인트로 간주하여 격리해 버리는 밀도 장벽 오작동을 겪는다. 파라미터 조절을 통해 eps=2.0, min_samples=10 설정에서 4개의 군집 형태를 유도하지만, 정상 상태 편향을 극복하기 어렵다.

• 계층적 군집화 및 최종 채택:

계층적 군집화는 Ward 연결법과 K=4 설정에서 실루엣 스코어 0.2234를 기록한다. 데이터 분포의 경계가 자잘하게 쪼개지는 경향을 보이나, 에러 격리 능력 자체는 유의미하게 유지된다. 종합적인 성능과 물리적 해석력을 기준으로 전류 변수 7개(Group A)에 K-Means(K=4)를 적용한 모델을 최종 비지도 군집 모델로 선정한다.

5.3 군집 분석 결과의 물리적 의미 복원 및 고장 연계

최종 선정된 다차원 K-Means 모델의 4개 군집을 데이터 상의 실제 정상/고장 라벨과 매핑하여 비지도 학습 결과에 공학적 타당성을 부여한다.



• 도출된 4대 군집의 물리적 상태 해석:

- **Cluster 0 (정상 대기):**
그리퍼와 관절 전류가 기저 전력 대역에서 안정적으로 유지되는 무부하 정상 정지 상태이다.
- **Cluster 1 (정상 파지 및 이송):**
높은 그리퍼 작동 전류 대역을 형성하며 실하중 물체를 안정적으로 움켜쥐고 이송하는 작업 상태이다.
- **Cluster 2 (관절 과부하 상태):**
그리퍼 자체의 부하는 낮으나 어깨 J1과 팔꿈치 J2 관절의 기저 전류가 비정상적으로 급상승하는 고부하 군집이다.
- **Cluster 3 (전이 동작 상태):**
그리퍼가 열리고 닫히는 움직임 전이 구간을 나타낸다.

• 비지도 군집 기반의 선제적 고장 격리 실증:

- 군집별로 실제 로봇 고장 발생 빈도를 추적한 결과, 어깨와 팔꿈치 모터의 과부하 패턴을 담은 Cluster 2에서 전체 안전 보호 정지(Protective Stop) 에러 샘플의 14.34%가 밀집되어 발생함을 규명한다.
- 이는 지도학습 라벨 없이 오직 관절 전류의 분포 패턴만을 활용한 비지도 학습 모델이 로봇의 이상 과부하 징후와 보호 정지 위험 영역을 사전에 명확히 구획하고 예측할 수 있음을 입증한다.
- 대조 알고리즘인 계층적 군집화의 경우 Cluster 3 영역에서 보호 정지 에러 발생률이 17.90%로 탐지되며, DBSCAN의 경우 격리된 노이즈 군집(-1)에서 8.86%의 고장률을 포착하여 알고리즘 전반에서 비지도 기반 고장 격리의 타당성이 증명된다.

더욱 자세한 Clustering 과정은 아래의 보고서에 담겨있다.

https://github.com/kzming2007/UR3_Cobot_ML/blob/main/reports/Data_Clustering_Task_Report.pdf

6. 종합 결론

6.1 프로젝트 수행 과정 및 모델 구축 흐름

본 프로젝트는 고가의 추가 외부 센서 없이 협동로봇의 기본 내장 모터 신호만을 역추적하여 작동 상태를 모니터링하고 고장을 예측하는 소프트웨어 기반 가상 센싱 기술의 가능성을 검증한다. 프로젝트의 세 가지 머신러닝 태스크는 선형적인 나열을 넘어 동역학적인 계층적 규명 및 패러다임 전환이라는 유기적 인과관계로 연결된다.

• 회귀 분석의 한계와 분류로의 전환:

그리퍼 구동 전류를 연속형 수치로 예측하고자 다양한 선형 및 비선형 회귀 모델을 적용한다. 랜덤 포레스트 모델이 결정계수 0.58 수준으로 가장 좋은 성능을 나타내지만, 센서 신호의 비선형적인 역 T자형 분포와 이산적인 삼봉 분포라는 물리적 제약으로 인해 예측 성능 향상에 한계를 겪는다. 이에 따라 연속형 수치 예측이 아닌 그리퍼의 이산적 상태를 분류하는 것이 공학적으로 타당하다는 인사이트를 도출하고 시스템 고장 예측 분류 태스크로 문제를 재정의해보았다.

• 분류 모델의 고도화와 데이터 누수 방지:

안전 보호 정지와 파지 상실을 통합한 시스템 고장을 예측하기 위해 불균형 데이터 제어 기법을 적용한다. 이 과정에서 교차 검증 각 단계 내에 스케일링과 오버샘플링을 결합한 엄격한 파이프라인을 구축하여 검증 데이터가 사전에 유출되는 데이터 누수 오류를 원천 차단한다.

• 비지도 군집화를 통한 기계적 상태 식별 및 고장 역추적:

단순히 정상과 고장 상태를 분류하는 것에 그치지 않고, 다차원 관절 전류 정보만을 활용하여 로봇이 실제로 몇 가지 운영 상태로 구동하는지 비지도 군집화를 수행한다. 도출된 4개의 물리적 운전 군집을 실제 시스템 고장 로그와 매핑하여 추적한 결과, 어깨와 팔꿈치 모터의 과부하 패턴을 담은 2번 군집 내에서 전체 안전 보호 정지 에러 발생률의 14.34%가 밀집되어 발생함을 규명한다. 이는 비지도 학습 모델이 로봇의 이상 과부하 전조 징후를 사전에 격리할 수 있음을 입증하는 유기적 성과이다.

6.2 모델별 최종 성능 요약

세 가지 머신러닝 태스크에서 총 12개의 모델을 실험하였다. 각 태스크의 최종 결과를 아래 표에 정리한다.

Task	최고 모델	핵심 성능 지표	비고
Regression	Random Forest	$R^2 = 0.5848$, $MSE = 0.0023$	파생 변수 <code>is_stationary</code> 적용
Classification	Random Forest	Macro F1 = 0.80, ROC-AUC = 0.94	SMOTE 적용, FP 91% 감소(LR 대비)
Clustering	K-Means (K=4)	Silhouette = 0.38	4가지 물리적 운영 상태 도출

6.3 3개 Task를 관통하는 공통 발견

세 태스크는 서로 다른 목적(예측, 분류, 군집화)으로 수행되었지만, 독립적으로 동일한 결론에 수렴하였다.

① 전류(Current)가 로봇 상태를 설명하는 핵심 신호다

Task	근거
Regression	RF 피쳐 중요도 상위: Current J4, Current J5 → 그리퍼 인접 관절 전류가 Tool Current 예측의 핵심

Task	근거
Classification	RF 피쳐 중요도 상위: Current J3, Current J5, Current J2 → 고장 탐지에 전류가 압도적
Clustering	전류 7개(Group A)가 최적 피쳐 그룹 → 속도와 온도는 군집화에 오히려 노이즈로 작용

세 태스크가 각각 독립적으로 “전류 변수가 로봇의 상태를 가장 잘 설명한다”는 동일한 결론에 도달했다. 이는 특정 모델이나 실험 설정에 좌우되지 않는, 데이터 자체가 품고 있는 **객관적인 물리적 사실**이다.

② 이 데이터는 선형으로 풀 수 없다

Task	선형 모델	비선형 모델	개선 폭
Regression	LR: $R^2 = 0.23$	RF: $R^2 = 0.58$	약 2.5배
Classification	Logistic: $F1 = 0.51$	RF: $F1 = 0.80$	약 1.6배

두 태스크 모두에서 선형 모델이 현저히 낮은 성능을 보였다. Regression에서는 역 T자형 비선형 분포가, Classification에서는 정상/고장 데이터의 복잡한 중첩이 선형 경계의 한계를 드러냈다. 이는 로봇 센서 데이터가 단순한 비례 관계가 아닌, **특정 조건에서 급변하는 비선형적 특성**을 지니고 있음을 일관되게 증명한다.

③ Random Forest가 이 데이터에 가장 적합한 알고리즘이다

Regression(R^2 최고)과 Classification(F1, AUC 최고) 모두에서 Random Forest가 최고 성능을 기록했다. 복잡한 비선형 관계, 변수 간 다중공선성, 극심한 클래스 불균형이라는 이 데이터의 세 가지 난제를 양상을 기법이 가장 효과적으로 극복했기 때문이다.

6.4 Task 간 교차 검증

각 태스크의 결과가 다른 태스크의 발견을 뒷받침하는 교차 검증 관계를 아래에 정리한다.

Regression → Classification: 비선형 모델링 전략의 당위성 확보

Regression 과정에서 로봇 센서 데이터의 극단적인 비선형성(역 T자 분포, 삼봉 분포)을 규명했다. 이 발견은 Classification 태스크에서 처음부터 선형 모델의 한계를 예견하고 비선형 앙상블 모델에 집중하는 전략적 근거가 되었다. 실제로 선형 모델(Logistic Regression)은 F1 0.51에 그쳤고, 비선형 모델(RF)은 F1 0.80을 달성하여 이 전략이 유효했음을 입증했다.

Classification → Clustering: 핵심 피쳐 그룹 선정의 근거

Classification의 피쳐 중요도 분석에서 전류(Current) 변수가 고장 예측의 핵심 신호임을 확인했다. 이 결과를 바탕으로 Clustering에서 전류 변수만으로 구성된 Group A를 기본 피쳐 그룹으로 설정했고, 실제로 Group A가 Group B(전류+속도), Group C(전체)보다 더 높은 실루엣 점수와 물리적으로 해석 가능한 군집 결과를 산출했다.

Clustering → Classification: 고장의 물리적 실체 규명

K-Means가 도출한 4개 군집 중 **Cluster 2(과부하 군집)**에서 Classification의 타겟이었던 Protective Stop 에러가 **14.34%** (K-Means), **17.90%** (Hierarchical Ward)의 비율로 집중 발생했다. 이는 Classification이 “고장/정상”이라는 이진 라벨만으로 예측하던 System Failure의 **물리적 실체**가, 어깨와 팔꿈치 관절에 과도한 전류 부하가 집중되는 특정 운영 상태임을 비지도 학습으로 독립 검증한 결과이다.

6.5 실무적 시사점

• 경제적이고 직관적인 가상 센싱 구현:

로봇에 고가의 물리적인 외부 센서를 추가 장착하지 않고, 제어기 내부에 기록되는 6개 관절의 전류, 속도, 온도 신호만으로 실험 하중과 구동 고장 상태를 정밀하게 진단할 수 있음을 계량적으로 실증한다.

• 도메인 지식 기반의 통계적 타당성 검증:

단순히 알고리즘의 통계적 정량 성능에만 매몰되지 않고, 다중공선성이나 실루엣 점수의 변화 원인을 로봇의 중력 보상 토크 및 손목 관절의 수평 유지 동역학 메커니즘과 연계하여 해석함으로써 머신러닝 분석에 뚜렷한 공학적 신뢰성을 부여한다.

따라서 본 프로젝트의 분석 결과를 실제 산업 현장에 적용한다면, 다음과 같은 시스템을 구축할 수 있다.

1. **실시간 상태 모니터링:** K-Means 모델을 탑재하여 관절 전류 데이터를 실시간으로 입력받고, 현재 로봇이 4가지 운영 상태 중 어디에 위치하는지를 대시보드에 시각화한다.

2. **사전 경고 시스템:** 로봇이 과부하 군집(Cluster 2)에 진입하는 빈도가 증가하면, Protective Stop 에러가 발생하기 전에 사전 경고 알람을 발생시킨다.
3. **이상치 탐지:** DBSCAN 기반의 이상 탐지 모듈을 병행하여, 어떤 군집에도 속하지 않는 극단적인 센서 패턴(노이즈 포인트)을 즉시 감지한다.

이러한 **K-Means(상태 분류) + DBSCAN(이상치 탐지)**의 2중 안전망 구조는, 추가적인 외장 센서 없이 로봇 내부의 관절 전류 데이터만으로 구현할 수 있다는 점에서 현실적인 경제성을 갖는다.

6.6 한계점 및 향후 과제

한계점	원인	개선 방향
Regression: R^2 0.6의 벽	관절 센서만으로는 그리퍼의 파지 여부를 100% 결정할 수 없는 물리적 한계	그리퍼 자체의 접촉 센서, 슬립 센서 등 말단 센서 데이터 추가
Classification: Recall 65%	정상/고장 데이터의 분포가 상당 부분 중첩	시계열 정보(이전 타임스텝의 전류 변화 추세) 활용, LSTM 등 시퀀스 모델 도입
Clustering: 경계 모호성	로봇 센서 데이터가 불연속적 군집이 아닌 연속적 스펙트럼을 형성	Fuzzy C-Means 등 소프트 군집화 기법으로 경계 유연화
공통: 정적 분석의 한계	현재 분석은 개별 시점(Row) 단위이며, 시간적 맥락을 고려하지 않음	슬라이딩 윈도우 기반 피쳐 추출 또는 시계열 딥러닝 모델 적용

- **시계열적 종속성의 반영 부재:**

로봇의 센서 로그는 시간에 따라 연속적으로 유기적 변화를 겪는 시계열 데이터이다. 그러나 본 분석은 각 초단위 로그 프레임을 개별 독립 샘플로 간주하여 학습을 진행한 한계가 존재한다.

- **데이터 과도 정제로 인한 고장 전조 소실:**

클래스 불균형을 해소하기 위해 통계적 오버샘플링을 거치는 과정에서, 정상에서 고장으로 전이되는 경계면 상의 미세한 전류 진동 정보가 노이즈로 오인되어 과도하게 정제될 우려가 있다. 로봇 공학 도메인에서 이러한 경계면 신호는 이상 동작의 귀중한 전조증상이므로 원천 정보 보존을 위한 세심한 전처리 통제가 요구된다

6.7 맺음말

본 프로젝트는 UR3 협동로봇의 관절 센서 데이터를 대상으로 회귀, 분류, 군집화의 3가지 머신러닝 태스크를 수행하며, 총 12개의 모델을 실험하고 비교 분석하였다.

단순히 높은 성능 수치를 추구하기보다, **“왜 이런 결과가 나왔는가”**를 데이터의 물리적 특성에 기반하여 해석하는 과정에 집중하였다. 그 결과, 역 T자형 비선형 분포, 삼봉 타겟 분포, 중력 보상 메커니즘, 연속적 스펙트럼 등 로봇 센서 데이터의 본질적인 구조를 규명할 수 있었다.

세 태스크가 독립적으로 도출한 공통 결론 — **전류가 핵심 신호이며, 비선형 양상을 모델이 가장 적합하다** — 은 특정 실험 설정에 종속되지 않는 일반화 가능한 발견이다. 이러한 발견은 협동로봇의 예지 정비 시스템 구축에 실질적인 기초 자료로 활용될 수 있을 것으로 기대한다.